ARI3129: Advanced CV for AI

Group 2

Daniel Borg Finn	0115405L	**Daniela Curmi**	0058805L
Matthew Pirotta	0297505L	**Matthew Vella**	0099105L

Github: https://github.com/Matthew-Pirotta/Advanced-Computer-Vision
Dataset: https://drive.google.com/drive/u/1/folders/1bPxVoXl6eLSzq9qq4dz779dkLnRC_sOA

Introduction **2**
Background on Techniques Used **2**
Object Detection 2
Attribute Classification 6
Data Preparation **8**
Dataset 8
Data Augmentation 14
Implementation of the Models **15**
Implementation of Object Detectors 15
Implementation of Classifiers 16
Evaluation of Object Detectors **17**
YOLOv8n 17
YOLOv11n 19
RF-DETR 21
Faster-RCNN 24
Evaluation of Attribute Classifiers **26**
ResNet-18 - Angle Classifier 26
ResNet-34 - Sign Shape Classifier 28
EfficientNet-B0 - Mounting Type Classifier 30
EfficientNet-B2 - Sign Condition Classifier 32
Final Group Analysis **35**
References and List of Resources Used **40**

Introduction

This project investigates how various computer vision techniques can be used to analyse and classify road traffic signs within Malta. In support of this, an image dataset: Maltese Traffic Signs Dataset (MTSD) was constructed, containing high quality images of signs in Maltese streets, where each image was annotated for both training and evaluation. This dataset targeted a defined set of sign categories, those being; Stop, No Entry (One Way), Pedestrian Crossing (Zebra Crossing), Roundabout Ahead, No Through Road (T-sign), and Blind-Spot/Convex Mirrors, where each of these signs were captured at various angles.

The attributes captured during annotation included viewing angle (Front, Side, Back), mounting type (Wall or Pole), sign condition (Good, Weathered, Heavily Damaged), and shape type (Circular, Square, Triangular, Octagonal, or Damaged). Label Studio was used both for capturing these annotations, as well as exporting to COCO and JSON formats into a combined dataset to ensure compatibility for the vision techniques used. Furthermore, this project applies models into two stages, first, traffic signs were first categorised using the following object detectors: YOLOv8, YOLOv11, Faster R-CNN and RF-DETR, after which these attribute classifiers: ResNet-18, ResNet-34, EfficientNet-B0, and EfficientNet-B2 were then trained to predict the selected sign attributes.

Background on Techniques Used

Automatic traffic sign analysis is commonly decomposed into two complementary computer vision problems: object detection, which aims to localise and identify traffic signs within complex street scenes, and attribute classification, which focuses on inferring additional properties of detected signs, such as viewing angle, mounting type, physical condition, or shape. In this project, a modular, two-stage pipeline is adopted, where traffic signs are first detected and localised, and subsequently analysed by a dedicated model to classify a selected attribute. This separation of concerns allows each stage to be optimised independently, supports fair comparison across different model choices, and improves interpretability by clearly distinguishing between localisation and attribute-level reasoning.

Object Detection

Object detection in urban street environments presents several challenges, including varying illumination conditions, background clutter, occlusions, and significant scale variation caused by distance and camera perspective. In the context of traffic sign analysis, these challenges are further compounded by the relatively small size of signs within the image and their frequent appearance at oblique viewing angles.

Object Detection using YOLO

Modern one-stage detectors from the YOLO family address these issues by performing object localisation and classification jointly in a single forward pass, enabling both high detection accuracy and computational efficiency. YOLOv8 builds upon this paradigm through a series of architectural refinements that improve robustness and generalisation, making it well suited for traffic sign detection [4].

A key refinement introduced in YOLOv8 is the use of an anchor-free detection strategy. Unlike earlier anchor-based approaches that rely on predefined bounding box templates, anchor-free detection predicts object locations directly from feature map points [12]. This reduces sensitivity to dataset-specific object size distributions and removes the need for manual anchor tuning. For traffic signs, which can appear at widely varying scales depending on camera distance and viewpoint, this approach improves

generalisation and stabilises training, particularly when working with custom datasets of limited size. As a result, the detector is better able to localise small or distant signs without extensive hyperparameter optimisation.

In addition, YOLOv8 incorporates improved multi-scale feature aggregation, allowing the network to combine fine-grained spatial details with higher-level semantic context [12]. This capability is particularly important for detecting traffic signs, as accurate localisation often depends on subtle visual cues such as edges, shapes, and colour patterns, while contextual awareness helps reduce false positives arising from visually similar background elements. By strengthening information flow across feature scales, the model maintains sensitivity to small objects while preserving robustness to viewpoint and lighting variations.

Finally, YOLOv8 employs optimised loss functions that more effectively balance classification confidence and localisation accuracy during training. Precise bounding box prediction is critical in this project, as detection outputs are subsequently used to extract cropped sign regions for attribute classification. Improved localisation reduces background interference in these crops, thereby supporting more reliable downstream analysis [8]. Overall, the architectural design of YOLOv8 aligns well with the requirements of traffic sign detection in real-world environments, providing an effective and scalable foundation for subsequent attribute classification tasks.

Progression from YOLOv8 to YOLO11

The YOLO architecture has continued to evolve beyond YOLOv8, with subsequent versions addressing specific limitations in detection pipelines. YOLOv9 introduces Programmable Gradient Information (PGI), which addresses information loss that occurs as data passes through successive network layers. PGI maintains auxiliary gradient pathways that preserve essential features throughout the network depth, ensuring reliable gradient generation during training [9]. For traffic signs, this preservation of fine-grained details such as edge sharpness and colour boundaries is important, as these features can otherwise degrade when propagating through deep feature hierarchies. YOLOv9 also introduces the Generalized Efficient Layer Aggregation Network (GELAN), which allows flexible integration of computational blocks while optimizing parameter utilization. This enables the model to balance representational capacity with computational efficiency, adapting to varying complexity requirements across different detection scenarios.

YOLOv10 removes the need for non-maximum suppression (NMS) by employing dual label assignments during training. The model uses a one-to-many head to generate multiple predictions per object during training, providing rich supervisory signals that improve learning. During inference, a one-to-one head produces a single optimised prediction per object, eliminating the post-processing bottleneck of NMS while maintaining prediction quality [8]. This architecture reduces inference latency, which is beneficial for real-time traffic sign detection systems where processing speed is critical.

In addition to NMS-free detection, YOLOv10 incorporates several efficiency-driven design improvements. The classification head employs depth-wise separable convolutions, which reduce computational overhead while preserving discriminative capability. Spatial-channel decoupled downsampling separates spatial resolution reduction from channel modulation, minimizing information loss during feature compression. The model also uses large-kernel convolutions to expand the receptive field, enabling broader contextual awareness that can help distinguish traffic signs from visually similar background elements.

YOLOv11 was used alongside YOLOv8 in this project. YOLOv11 introduces further improvements over YOLOv10 in the backbone and neck architecture [2]. These improvements enhance feature extraction

and multi-scale aggregation capabilities. The model also demonstrates improved parameter efficiency, with YOLO11m using 22% fewer parameters than YOLOv8m. By evaluating both YOLOv8 and YOLOv11 on the same traffic sign dataset, this project can assess whether the accumulated architectural improvements translate to measurable gains in traffic sign detection accuracy.

Object Detection using RF-DETR

RF-DETR is a transformer-based object detector derived from the DETR (DEtection TRansformer) framework, which constructs object detection as a direct set prediction problem, eliminating the need for handcrafted components such as anchor boxes and NMS [1]. RF-DETR is designed to balance accuracy and real-time latency through weight-sharing neural architecture search (NAS) rather than a single fixed architecture. Building on LW-DETR, RF-DETR introduces targeted changes to the backbone, attention mechanisms, decoder supervision, and query handling to improve both robustness and efficiency on real-world detection tasks, making it well-suited to traffic sign detection, where objects often appear in varied visual conditions.

For the backbone, RF-DETR replaces LW-DETR's CAEv2 encoder with a DINOv2-pretrained Vision Transformer (ViT), improving generalisation to limited and domain-specific datasets, while enhancing sensitivity to fine-grained visual patterns such as sign symbols, edges, and colour contrasts. The backbone outputs are passed through a lightweight multi-scale projector that produces feature maps at different resolutions, improving small-object localisation by preserving fine spatial detail (useful for background signs) without incurring the high computational cost of traditional feature pyramids. Layer normalization is used throughout the projection layers to ensure stable optimisation under small batch sizes, which is common when training on high-resolution datasets [6].

To balance global context and computational efficiency, RF-DETR interleaves windowed and non-windowed self-attention within the encoder. Windowed attention limits computation to local regions, which is effective for capturing local sign structure, while periodic global attention layers preserve long-range context, such as road layout [6]. This combination is particularly beneficial for traffic sign detection, as it allows the model to disambiguate signs from visually similar background objects by using local feature representation with global contextual awareness. RF-DETR follows a DETR-style encoder-decoder design with deformable cross-attention in the decoder, however detection losses are applied at every decoder layer rather than only the final layer, allowing predictions to be progressively refined across layers [1]. This enables decoder truncation at inference time, allowing the model to reduce latency without retraining.

During training, predictions are matched to ground-truth objects using bipartite Hungarian matching, enforcing a strict one-to-one correspondence that prevents duplicate detections and stabilises optimisation in scenes containing multiple nearby signs. Object queries are a core architectural component and are optimised through NAS. Each learnable query represents a potential object hypothesis and is responsible for predicting at most one traffic sign. RF-DETR further introduces query dropping at inference time, where low-confidence queries are removed based on predicted class scores [6]. Given that traffic scenes typically contain relatively few signs per image, this mechanism reduces unnecessary computation while preserving detection accuracy, allowing the model to adapt naturally to low object density scenarios.

Recent variants of the YOLO family have introduced architectural refinements, advanced data augmentation strategies, and improved training techniques to enhance detection performance while preserving fast inference speeds [2,4,8]. Despite their efficiency, most YOLO-based detectors rely on NMS to remove duplicate predictions, which introduces additional post-processing latency. In contrast, DETR eliminates hand-crafted components such as anchor boxes and NMS [1]. However, early DETR

variants achieved strong detection accuracy at the expense of high computational cost, limiting their applicability in real-time settings. More recent approaches, including RT-DETR and LW-DETR, have demonstrated that transformer-based detectors can be adapted for real-time performance without sacrificing accuracy. Building on these advances, RF-DETR further improves the accuracy-latency trade-off, as seen in Figure 1.

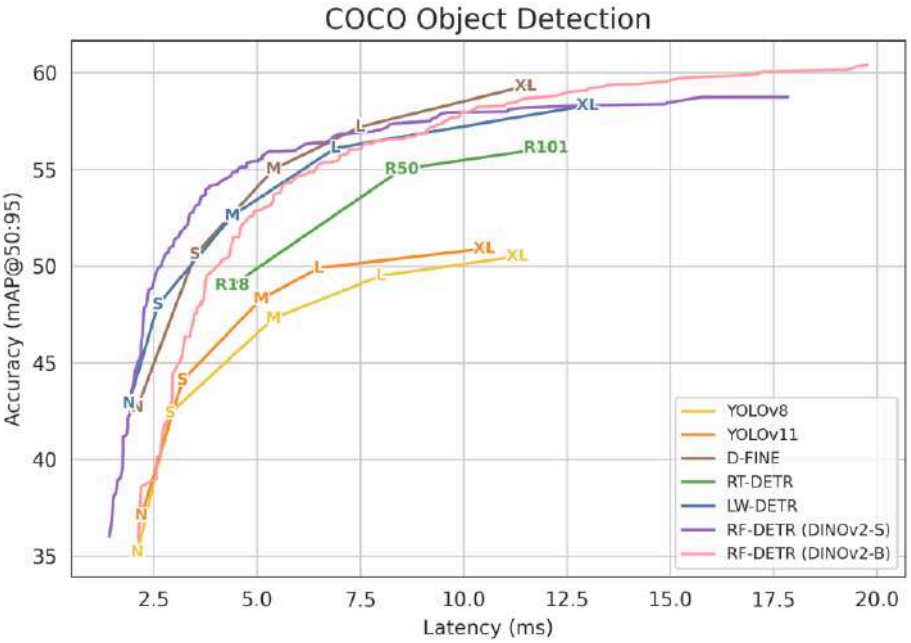


Figure 1: Accuracy-Latency Pareto Curve by Robinson et al. [6]

Object Detection using Faster-RCNN

Faster R-CNN is a two-stage object detection method that was released out of the original R-CNN line, where the main idea is to first generate a small list of candidate regions that might contain objects, then perform a more detailed classification and box refinement on those candidates. This general strategy came from earlier R-CNN work, where the core efficiency idea was to run a convolutional neural network to produce a feature map, which can be viewed as a compressed representation of the image that retains information related to recognizing objects. Furthermore, the first R-CNN approach involved producing region proposals from an external algorithm, where each of these regions was processed independently, leading to high accuracy but poor speed due to the model repeating expensive computations many times per image [11].

Fast R-CNN improved this pipeline by computing the CNN feature map just once and using Region of Interest (RoI) pooling to extract a fixed-size feature vector for each candidate region directly from the feature map. This allowed classification and bounding-box regression to be trained more efficiently, however, this approach still relied on an external region proposal step, meaning the overall runtime was still limited. Faster R-CNN was then developed to address this bottleneck by introducing the Region Proposal Network (RPN), a lightweight fully convolutional network that predicts object proposals from the feature map. Here, instead of running a separate algorithm, RPN uses the same feature map that the detector already computed, where it quickly predicts the candidate boxes directly from those features. In other words, the network looks at an image just once, then reuses that information to suggest where objects might be [5].

The RPN does this by checking many pre-set box shapes called anchors across the image, where for each anchor, it predicts whether it likely contains an object and small adjustment to make the box fit better. Because this happens on the shared feature map it is fast and efficient, producing a good number

of candidate regions for the second stage, where the detector extracts features for each proposal and then predicts the object class [5].

Attribute Classification

Following the localisation of traffic signs by the object detector, the classification of sign attributes can be formulated as a standard image classification problem applied to cropped sign regions.

Mounting Type Classification using EfficientNet-B0

Mounting types for street signs can be predicted using an image classifier built on EfficientNet-B0, where the goal is to assign each detected sign to the following categories, pole-mounted and wall-mounted. This is done by learning visual evidence such as the presence of poles or brackets, as well as the sign's placement in relation to its environment. Specifically, EfficientNet-B0 is the smallest baseline in the EfficientNet series and was developed to provide feature extraction with relatively low demand of computation. The key idea behind it is compound scaling, where the model's depth, width and resolution are increased at the same time, helping the network use parameters more efficiently, effectively making it a solid choice when memory or inference speed are important [7].

Typically, EfficientNet-B0 is applied to cropped sign regions, that way it can concentrate on mounting-related details instead of noisy background details. In addition to this, keeping detection and attribute classification separate also makes it easier to evaluate and tune each component more independently, further highlighting EfficientNet-B0 as high performing in both accuracy and efficiency [7].

Sign Condition Classification using EfficientNet-B2

In this project, the sign condition attribute is predicted using a convolutional neural network based on the EfficientNet-B2 architecture. Sign condition classification is performed over three condition states, including good, weathered, and heavily damaged. For the scope of this task, rather than using full-sized images, the classifier takes cropped sign regions as input. Hence, irrelevant background information is removed, allowing the CNN to focus on sign condition features.

EfficientNet models are built using compound scaling, which jointly scales network depth, width, and input resolution in a principled manner, as seen in Figure Equation 3. Hence, all EfficientNet models are scaled from the baseline EfficientNet-B0 model using a different compound coefficient φ. EfficientNet-B2 increases input resolution and feature depth relative to smaller variants, enabling it to capture fine-grained texture details and localised features that are indicative of sign condition, while remaining lightweight enough for practical deployment.

$$
\begin{aligned}
\text{depth: } & d = \alpha^{\phi} \\
\text{width: } & w = \beta^{\phi} \\
\text{resolution: } & r = \gamma^{\phi} \\
\text{s.t. } & \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2 \\
& \alpha \geq 1, \beta \geq 1, \gamma \geq 1
\end{aligned}
$$

Equation 3: compound scaling method equation used by [7]

EfficientNet-B2 is composed of a series of mobile inverted bottleneck convolution (MBConv) blocks with depthwise separable convolutions and squeeze-and-excitation (SE) attention. These components allow the model to efficiently model both spatial detail and channel-wise feature importance, which is

particularly beneficial for identifying condition-related features such as scratches, dents, rust, fading, or heavily damaged sign characteristics.

While EfficientNet-B0 uses the same architectural principles as EfficientNet-B2, it operates at a lower input resolution and with fewer parameters (5.3M), resulting in faster inference but reduced sensitivity to fine visual detail. As a result, EfficientNet-B2 outperforms B0 by achieving a 80.1% Top-1 Accuracy on ImageNet, at the cost of computational efficiency due to higher input resolution and an increase in the number of parameters used (9.2M) [7]. However, since sign condition classification depends on capturing fine-grained details, such as surface texture, rather than structural cues, EfficientNet-B2 was chosen due to its representational capacity.

Viewing Angle Classification using ResNet-18

In this project, the viewing angle attribute, categorised as front, side, or back view, is predicted using a convolutional neural network based on the ResNet-18 architecture. Residual Networks introduced the concept of residual learning through skip connections, allowing deep networks to be trained more effectively by mitigating vanishing gradient issues [3]. This design principle has made ResNet-based models a common and reliable choice for a wide range of visual recognition tasks.

ResNet-18 is one of the lighter variants within the ResNet family and is characterised by a relatively low parameter count and efficient computational profile. These properties make it particularly suitable for applied classification tasks involving custom datasets of limited size, such as traffic sign imagery collected in real-world environments. Viewing angle classification depends primarily on global geometric and structural cues, including perspective distortion, visible sign backing, mounting orientation, and edge configuration, rather than highly fine-grained texture information. ResNet-18 provides sufficient representational capacity to capture these spatial features while reducing the risk of overfitting that may arise when using deeper architectures on modest datasets [3].

ResNet-18 also pairs naturally with YOLOv8 in a two-stage detection–classification pipeline. YOLOv8 provides accurate bounding boxes that isolate traffic signs from complex backgrounds, enabling the ResNet-18 classifier to focus exclusively on sign-centric visual information. This separation of detection and attribute classification reduces background interference and simplifies the learning problem for the classifier. Additionally, the lightweight nature of ResNet-18 complements the real-time design philosophy of YOLO-based detectors, resulting in a pipeline that is both computationally efficient and modular. Such detector–classifier combinations are commonly adopted in practical vision systems where interpretability, scalability, and independent evaluation of each stage are desirable.

Overall, ResNet-18 offers an effective trade-off between model complexity, generalisation performance, and computational cost. When used for viewing angle classification on detector-generated crops, it provides a robust and efficient solution that integrates seamlessly with YOLOv8-based traffic sign detection.

Sign Shape Classification using ResNet-34

The sign shape attribute is predicted using a convolutional neural network based on the ResNet-34 architecture. Sign shape classification involves distinguishing between geometric categories such as circular, triangular, rectangular, and octagonal signs based on boundary contours, corner configurations, and aspect ratios.

ResNet-34 extends ResNet-18 with additional residual blocks, increasing network depth from 18 to 34 layers while maintaining the residual learning framework through skip connections [3]. This increased

depth allows the network to develop more refined feature hierarchies for capturing geometric properties at multiple levels of abstraction. Despite its greater capacity, ResNet-34 remains computationally efficient relative to deeper variants such as ResNet-50 or ResNet-101.

As with viewing angle classification, ResNet-34 operates on bounding box crops generated by YOLOv11, allowing the classifier to focus on sign-centric geometric features without background interference. This modular separation between detection and classification enables independent evaluation and optimisation of each pipeline stage. Overall, ResNet-34 provides sufficient capacity for shape classification while maintaining computational efficiency suitable for practical deployment.

Data Preparation

Dataset

The dataset consists of 630 traffic sign images containing 770 sign annotations, collected manually across nine localities in Malta: Ħ'Attard, Ta' Qali, Mosta, Ħal Lija, Imrieħel, St Paul's Bay, Senglea, St Venera, and San Ġwann. Images were captured during daytime under varying lighting conditions, viewing angles, and distances to reflect realistic street-level scenarios. The average of 1.22 annotations per image indicates that most images contain a single sign, though some scenes include multiple signs. Table 1 shows the contribution breakdown across the four team members.

Contributor	Images	Annotations
Daniel Borg Finn	151	154
Daniela Curmi	165	243
Matthew Pirotta	164	204
Matthew Vella	150	169
Total	**630**	**770**

Table 1: Dataset contribution by team member

Before annotation, each image was manually reviewed and all faces and number plates were blurred or obscured to ensure privacy and compliance with GDPR. Data annotation was then performed using Label Studio. Traffic signs were labeled with bounding boxes for object detection according to sign type, and classified according to viewing angle, mounting type, sign condition, and sign shape.

The following sections present the dataset composition, class distributions, and visual characteristics through exploratory analysis.

Dataset Distribution

The distribution of annotations was analyzed across sign type, viewing angle, mounting type, condition, and sign shape. These distributions reflect the composition of the collected dataset and the prevalence of different sign categories in the captured scenes.

Sign Type Distribution

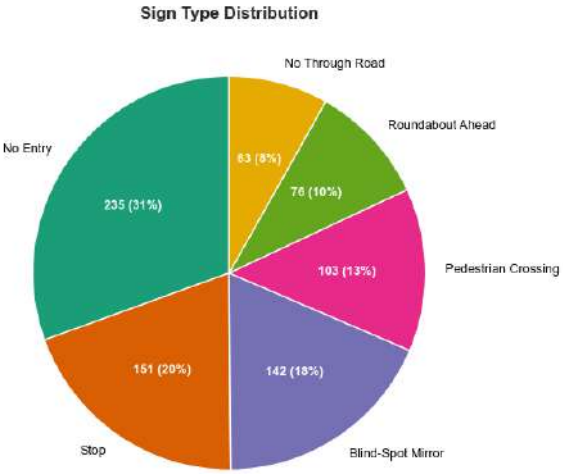


Figure 2: Distribution of traffic sign types in the dataset

As shown in Figure 2, the dataset exhibits class imbalance across sign types. No Entry signs constitute the largest class at 31%, followed by Stop signs at 20%. Blind-Spot Mirror (Convex) signs represent 18% of the dataset, a relatively high proportion that reflects their prevalence in Maltese road infrastructure. Pedestrian Crossing, Roundabout Ahead, and No Through Road signs form minority classes at 13%, 10%, and 8% respectively. This imbalance reflects real-world sign frequency but may affect model performance on underrepresented classes.

View Angle Distribution

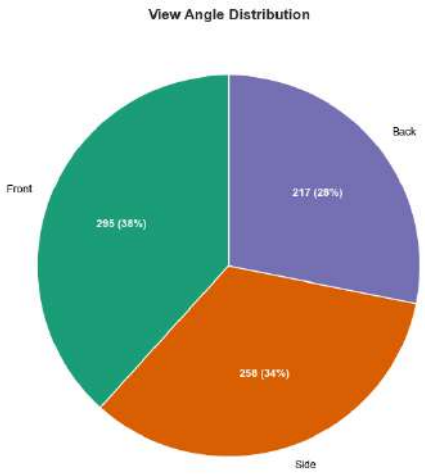


Figure 3: Distribution of viewing angles in the dataset

Viewing angles are relatively balanced across the dataset (Figure 3), with front views at 38%, side views at 34%, and back views at 28%. Although front, side, and back images were captured for each sign whenever possible, additional signs appearing in the background were also annotated, contributing to the overall distribution and providing varied multi-sign scenes. The lower proportion of back views is partly due to wall-mounted signs being inaccessible from behind.

Mounting Type Distribution

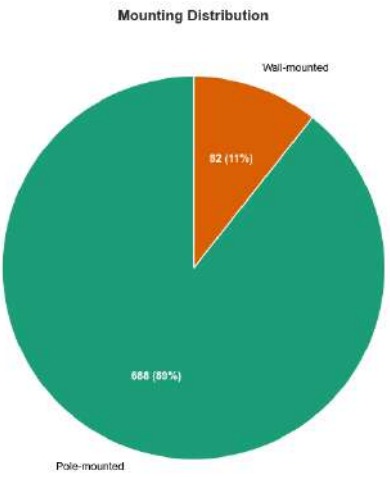


Figure 4: Distribution of mounting types in the dataset

The dataset is heavily skewed toward pole-mounted signs at 89%, with wall-mounted signs representing only 11% (Figure 4). This reflects typical mounting practices but limits model exposure to wall-mounted configurations, which may affect detection performance in scenarios where wall-mounted signs are more common.

Condition Distribution

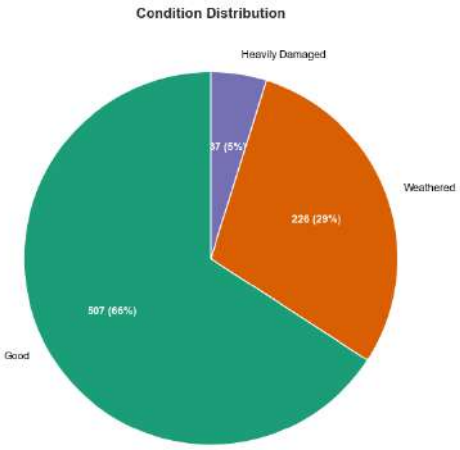


Figure 5: Distribution of sign conditions in the dataset

Most signs are in good condition at 66%, while weather-affected signs account for 29% and heavily damaged signs represent only 5%. This distribution (Figure 5) aligns with typical maintenance practices where most signs remain functional and readable. The small proportion of heavily damaged signs may limit the model's ability to handle severe degradation.

Sign Shape Distribution

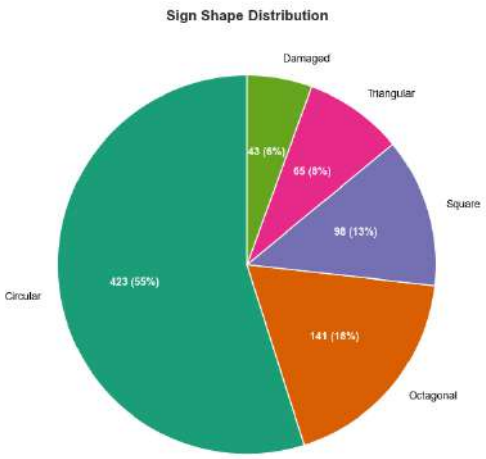


Figure 6: Distribution of sign shapes in the dataset

Figure 6 shows that circular signs represent the largest shape category at 55% since it encompasses No Entry, Blind-Spot Mirror, and Roundabout Ahead signs. Octagonal signs account for 18%, matching the Stop sign distribution. Square and Triangular signs form minority classes at 13% and 8% respectively. The Damaged category at 6% represents signs where physical damage affects shape characterization.

Dataset Attribute Analysis

To better understand the relationships between sign attributes and their potential impact on model training, cross-attribute analysis was performed across sign type, condition, mounting type, viewing angle, and shape.

Sign Type vs Condition

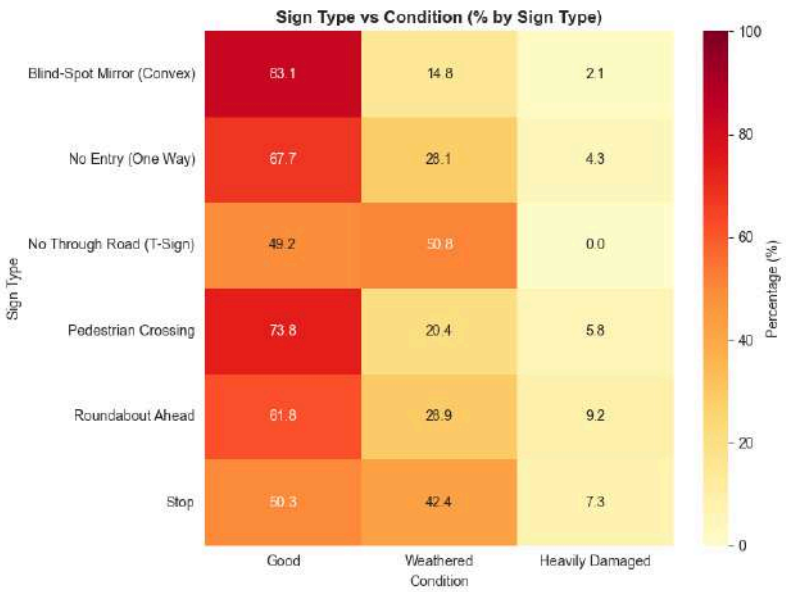


Figure 7: Distribution of sign condition across sign types

As shown in Figure 7, condition distribution varies significantly across sign types. Blind-Spot Mirror signs show the best condition, as these are reflective convex mirrors made out of glass or durable plastic

rather than painted metal signs, making them less susceptible to weathering and physical damage. Stop signs and No Through Road signs show the highest weathering rates, with roughly half in weathered condition. No Through Road signs notably show no heavy damage despite high weathering, suggesting they degrade gradually but remain structurally intact. Roundabout Ahead signs show the highest heavy damage rate, nearly double that of other sign types, which may reflect placement in high-wear locations or greater vulnerability to environmental damage. Overall, the majority of signs remain in good condition, while weathered signs are moderate and heavily damaged signs are rare.

Sign Type vs Mounting Type

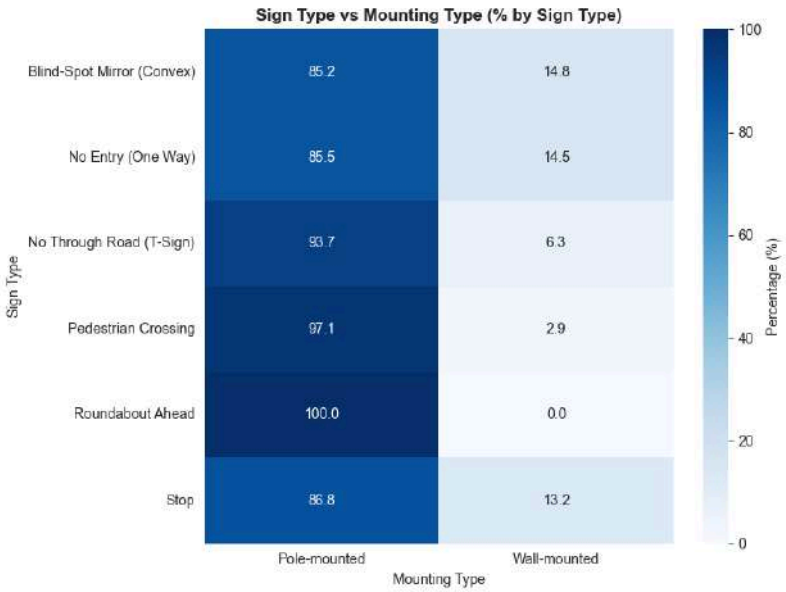


Figure 8: Mounting type distribution by sign type

The distribution of mounting type (Figure 8) reflects functional placement requirements. Roundabout Ahead signs are exclusively pole-mounted, while Pedestrian Crossing and No Through Road signs are predominantly pole-mounted. These warning and information signs require roadside visibility from a distance. Regulatory signs such as Stop, No Entry, and Blind-Spot Mirror show greater mounting diversity, as they can be placed at intersections or building entries where wall-mounting is practical.

Sign Type vs Viewing Angle

Figure 9: Viewing angle distribution across sign types

Figure 9 demonstrates that viewing angle distribution is consistent across all sign types, with roughly 28-30% back views, 35-41% front views, and 29-37% side views. This reflects the data-collection approach, where front, side, and back images were captured for each sign when possible, and background signs were also annotated when present. The slight front-view bias is consistent across all sign types, which indicates that no systematic bias was introduced during data collection.

Sign Type vs Shape

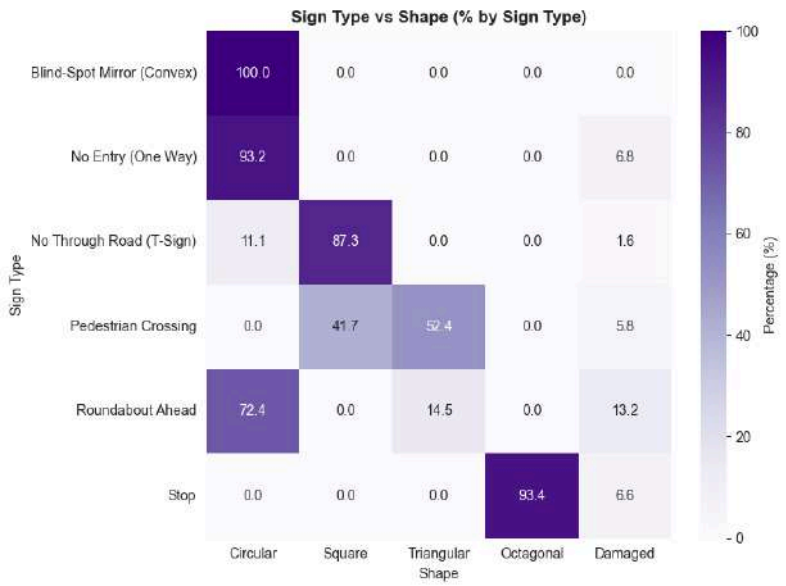


Figure 10: Shape distribution by sign type

Shape distribution (Figure 10) shows both expected mappings and some ambiguities. Blind-Spot Mirror signs are entirely circular, while Stop signs are predominantly octagonal with a small proportion classified as damaged when shape is compromised. No Entry signs are mostly circular with some damaged classifications. Pedestrian Crossing signs are split between triangular and square shapes, since Malta has two distinct pedestrian crossing sign types, triangular warning signs and rectangular information signs. These appear roughly equally in the dataset.

No Through Road signs are predominantly square, though some are classified as circular. Roundabout Ahead signs are primarily circular but also include triangular and damaged classifications. These cases were manually verified to confirm they were actual instances rather than annotation errors. The damaged shape classifications appear most frequently on sign types that also show higher rates of physical damage in the condition analysis, suggesting this category is applied when physical deterioration obscures the underlying geometric shape.

Data Augmentation

Data augmentation was employed to improve model generalisation and robustness by artificially increasing the variability of the training data while preserving semantic correctness. Given the limited size of the custom-collected traffic sign dataset and the diversity of real-world capture conditions, augmentation was considered essential to mitigate overfitting and to expose models to plausible variations in appearance, viewpoint, and illumination.

A task-aware augmentation strategy was adopted across the project. Rather than applying identical transformations to all models, augmentations were selected according to the role of each model within the pipeline, namely object detection and attribute classification, while adhering to a shared set of constraints to ensure realism. In all cases, transformations were designed to reflect variations that could realistically occur during image capture, such as small changes in camera position, scale, and lighting, while avoiding distortions that would alter the physical meaning of the traffic sign or invalidate its labels.

Geometric augmentations were used conservatively to reflect realistic variations in street-level image capture. Rotations, translations, and scale variations were applied to simulate differences in camera angle, distance, and framing, helping models become invariant to minor viewpoint changes without compromising the structural integrity of traffic signs. Horizontal flipping was permitted, as left–right reflection is generally valid for traffic signs, which are designed to be symmetric or equivalent under such transformations. In contrast, more aggressive geometric distortions, including strong perspective warping, excessive shearing, vertical flipping, as well as compound augmentations such as mosaic and mixup, were deliberately avoided, as they can introduce physically implausible sign appearances, conflict with attribute labels such as viewing angle, or degrade localisation accuracy for small objects.

Photometric augmentations were applied to model variability in lighting conditions, including changes in brightness, saturation, and colour balance. This is particularly relevant for outdoor scenes, where illumination can vary significantly due to time of day, weather conditions, and shadowing. By exposing models to such variations during training, robustness to lighting changes at inference time is improved.

For attribute classification, augmentations focused on preserving global structural cues relevant to the prediction task, such as sign orientation and visible backing, while still encouraging invariance to minor spatial and colour variations. For sign shape classification specifically, geometric augmentations included small rotations of up to ±10 degrees and horizontal flipping with 30% probability, accounting for slight variations in camera orientation while preserving the structural features necessary for shape discrimination. Moderate colour jittering was applied to improve robustness to changes in illumination and colour balance commonly encountered in outdoor environments, without altering the semantic interpretation of the sign. Cropping-based augmentations were used to simulate imperfect detections and framing variability, reinforcing robustness when operating on detector-generated crops.

Overall, the augmentation strategy prioritised realism, label consistency, and task relevance. By tailoring augmentation choices to the functional role of each model while maintaining a consistent design

philosophy, the project supports fair model evaluation, and ensures that performance gains arise from meaningful generalisation rather than unrealistic data transformations.

Implementation of the Models

Implementation of Object Detectors

YOLOv8n

YOLOv8 Nano was implemented using the Ultralytics framework and initialised with pretrained weights. The model was trained for 100 epochs on the custom traffic sign dataset, with a batch size of 8. Training was performed using early stopping with a patience of 20 epochs to mitigate overfitting.

YOLOv11n

YOLOv11 Nano was implemented using the Ultralytics framework with pretrained COCO weights. The model was trained over 100 epochs with images standardized to 640×640 resolution, batch size of 16, and early stopping patience of 20 epochs. Performance was evaluated on the test set using mAP@0.5, mAP@0.5:0.95, precision, recall, and inference speed metrics.

RF-DETR Nano

Before implementing the RF-DETR Nano model, the 'COCO-based_COCO' datasets' structure was modified to fit the requirements of the model, which expected three folders corresponding to each dataset split, each containing annotations stored in COCO format and the respective images. The RF-DETR Nano model is instantiated as a lightweight configuration that limits architectural capacity while retaining the core RF-DETR architecture, making it well suited for object detection on a fixed dataset of images. Initialising 'RFDETRNano' with 'num_classes=6' adapts the model's classification head to the traffic sign label set, categorised as Blind-Spot Mirror (Convex), No Entry (One Way), No Through Road (T-Sign), Pedestrian Crossing, Roundabout Ahead, Stop. In addition, disabling pretrained weights ensures that all parameters are learned directly from our sign dataset. The training configuration reflects the model's transformer-based architecture, since a moderate learning rate (2×10^{-4}) with weight decay supports stable optimisation, while warm-up over the first five epochs mitigates early training instability commonly observed in attention-based models. The model was trained over a 100 epochs, using a small batch size of 4 to accommodate high-resolution imagery and GPU memory constraints, and early stopping via a patience of 20 epochs helps prevent overfitting of the limited sign dataset. Overall, this setup leverages RF-DETR Nano's architecture to produce a detector well suited for accurate, low-latency traffic sign detection.

Faster R-CNN

Faster R-CNN was trained in this implementation using annotations stored in COCO format with the torchvision FasterRCNN_MobileNetV3_Large_FPN model as the backbone of the detector. Here a custom dataset class, 'CocoIndex' parsed the Json files and indexed the images/annotations, then mapped category IDs to contiguous labels compatible with Faster R-CNN via 'CatID_ToLabel'. All images were standardised to 640x640 using letterboxing in 'ResizeImage()', where light data augmentation was applied later on inside 'LoadSample()', including horizontal flips, colour jitter, or blurring.

Implementation of Classifiers

ResNet-18 (Viewing Angle)

A pretrained ResNet-18 was fully fine-tuned end-to-end, with the final classification layer replaced to match the number of viewing-angle classes. Full fine-tuning was chosen instead of freezing the backbone to allow pretrained features to adapt to the fine-grained geometric and perspective cues required for viewing-angle classification, which differ substantially from the semantic object categories learned during ImageNet pretraining.

ResNet-34 (Sign Shape)

A pretrained ResNet-34 was fully fine-tuned for five-class shape classification (Circular, Damaged, Octagonal, Square, Triangular), with the final layer replaced to match the number of classes. The classification dataset was constructed by cropping individual signs from YOLO bounding boxes with 5% padding and EXIF rotation correction. Training used Adam optimization with weighted cross-entropy loss to address class imbalance, early stopping with patience of 5 epochs, and standard augmentation including random flips, rotations, and color jittering. Model performance was assessed using accuracy, precision, recall, F1-score, and confusion matrices.

EfficientNet-B0 (Mounting Type)

EfficientNetB0 was trained in this implementation using COCO-format annotations, where COCOIndex parsed the JSON files and each image was assigned a single class label via majority vote over its bounding-categories, mapped to contiguous indices. Here the custom 'COCOMountingClassificationDataset' cropped images around the ground-truth boxes using 'CropByGT_Boxes() to 224x224, with light augmentation during training including, flips or random resized crops. Training was performed in main(), where a pretrained efficientnet_b0 had its classifier head replaced for the two mounting classes, where performance was then reported with TorchMetrics, including accuracy, precision/recall/F1. A Confusion matrix and a per-class report was also created to give further information on class-wise behaviour and misclassifications, highlighting where the model most frequently confused pole-mounted and wall-mounted signs.

EfficientNet-B2 (Sign Condition)

The sign condition classifier is implemented using EfficientNet-B2 using default weights for fine-tuning during training, replacing the final classification layer to predict three condition classes (i.e. Good, Weathered, and Heavily Damaged) while retaining the original backbone for feature extraction. The model is trained end-to-end using cross-entropy loss and the AdamW optimiser with weight decay, combined with cosine annealing to gradually reduce the learning rate and encourage stable convergence. The model was trained over a total of 10 epochs, however training stopped at epoch 6, since early stopping via a patience of 3 was employed to mitigate overfitting given the limited dataset size. Unlike common transfer-learning pipelines, the backbone isn't frozen during training; instead, all layers are fine-tuned from the beginning of the training process. This design choice allows the network to recalibrate its representations to these fine-grained appearance variations, improving discriminative capacity without the need for backbone freezing and unfreezing.

Evaluation of Object Detectors

Mean Average Precision (mAP) is a standard metric used to evaluate object detection performance by measuring how accurately a model detects and localises objects. mAP@0.5 considers a detection correct if the predicted bounding box overlaps the ground-truth box by at least 50%, providing a relatively lenient measure of detection accuracy. In contrast, mAP@0.5:0.95 averages performance over multiple overlap thresholds from 0.5 to 0.95, offering a more rigorous assessment of both classification and localisation quality.

YOLOv8n

Figure 11 shows the detection performance of the YOLOv8-nano traffic sign detector over training, measured using mAP@0.5 and mAP@0.5:0.95. Both metrics increase rapidly during the initial training phase, with mAP@0.5 rising from approximately 0.20 to over 0.55 within the first 10 epochs, indicating that the model quickly learns to localise traffic signs and separate them from the background.

After the initial phase, performance improves more gradually. Between epochs 10 and 30, mAP@0.5 stabilises around 0.68–0.72, while mAP@0.5:0.95 continues to increase toward 0.50. Minor oscillations are observed throughout training, reflecting batch-level variation and data augmentation effects, but no sustained drops in performance occur. Beyond approximately epoch 30, mAP@0.5 plateaus at around 0.72–0.74, whereas mAP@0.5:0.95 continues to improve slightly before converging near 0.56–0.58. This indicates that while detection accuracy saturates relatively early, bounding box precision continues to improve with additional training. Overall, the stable convergence and final performance confirm that the YOLOv8 detector provides reliable and sufficiently precise localisation for downstream attribute classification.

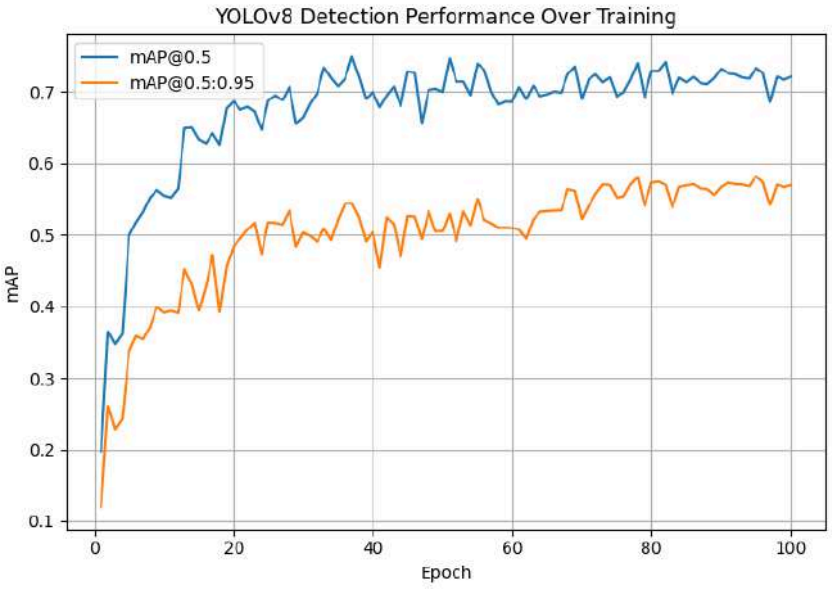


Figure 11: YOLOv8 sign detector mAP@X performance over training epochs

Figure 12 presents the precision–recall (PR) curves for the YOLOv8 traffic sign detector across individual sign classes, together with the aggregated performance over all classes. The detector achieves an overall mAP@0.5 of 0.723, indicating a good balance between precision and recall and stable behaviour across the dataset, as reflected by the smooth all-classes PR curve.

While some degree of class imbalance is present, it is not the sole determinant of detection performance. Although frequently occurring classes such as No Entry and Stop exhibit stable precision–recall behaviour (0.86 and 0.71 respectively), the No Through Road (T-Sign) class achieves the highest

average precision of 0.98 despite having the fewest samples. This suggests that visual distinctiveness plays a significant role in model performance. The T-Sign's consistent geometric structure and composition of clearly defined rectangular regions with contrasting colours make it easier for the detector to learn and reliably identify, even with limited training data.

In contrast, the Pedestrian Crossing class shows markedly poorer precision–recall performance of 0.37. Despite having a moderate number of samples, its precision drops sharply as recall increases, indicating a higher rate of false positives. This behaviour is likely influenced by high intra-class variability since pedestrian crossings come in two forms of rectangular information signs as well as rectangular warning signs. In addition to visual similarity to other sign types, as well as the strong bias toward circular sign shapes in the dataset.

Overall, the PR curves demonstrate that detection performance is influenced by both dataset composition and intrinsic visual characteristics of each sign type. Visually distinctive classes achieve high precision across a wide recall range, whereas more ambiguous classes remain challenging. These findings suggest that further improvements could be achieved through targeted data augmentation or additional data collection for underperforming classes, rather than relying solely on increasing overall dataset size.

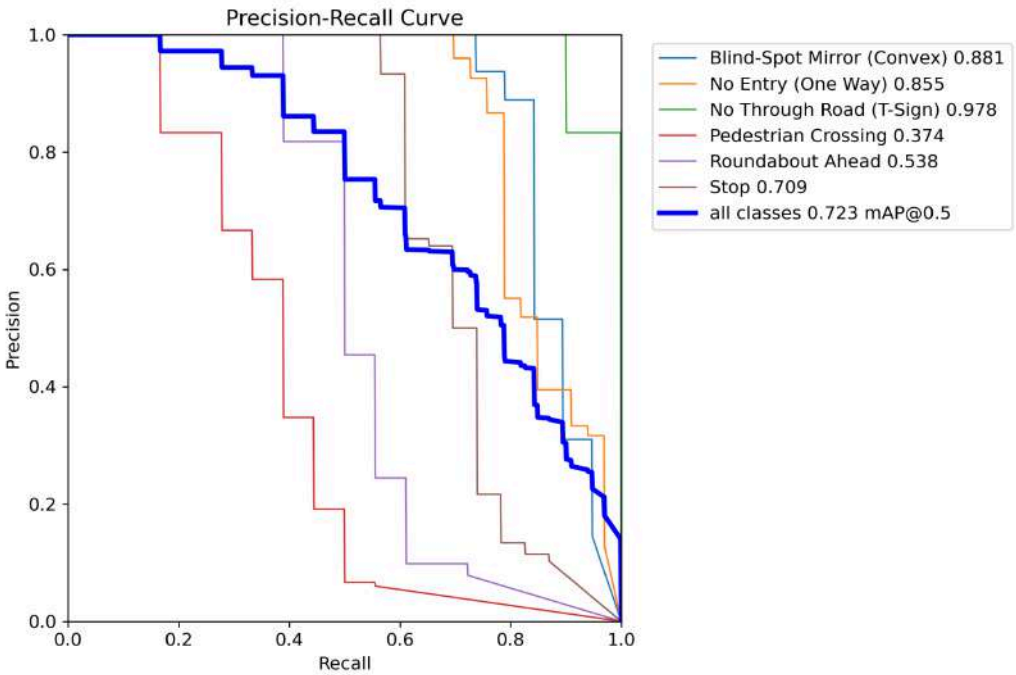


Figure 12: YOLOv8 sign detector Precision-Recall Curve for the different sign classes and overall

Figure 13 compares ground truth annotations with YOLOv8 predictions across representative street scenes. The detector successfully localises and correctly classifies multiple sign types, including No Entry, Blind-Spot Mirror (Convex), and Stop, under varying lighting conditions, scales, and background complexity. Predicted bounding boxes generally align closely with the ground truth, indicating accurate localisation even for small or partially occluded signs. However, in the bottom-left image, although the convex mirror is correctly detected, the associated confidence score is relatively low (0.5), suggesting reduced certainty under challenging visual conditions. Additionally, in the top-left image, the detector incorrectly classifies a No Waiting sign, which is not included in the training dataset, as a blind-spot mirror, albeit with a low confidence score of 0.4. This indicates that the model recognises the presence of a circular sign but lacks the semantic context to correctly distinguish unseen sign types, highlighting a limitation when encountering out-of-distribution objects. Overall, these examples illustrate that while the

detector performs reliably on known classes, its confidence and classification accuracy can degrade in ambiguous or novel scenarios.

Figure 13: YOLOv8 sign detector detection examples ground truth (left) and prediction (right)

YOLOv11n

The YOLO11n detector was trained for 80 epochs on the traffic sign dataset. Figure 14 shows training performance measured by mAP@0.5 and mAP@0.5:0.95. Both metrics start at approximately 0.18 and 0.14 respectively and rise sharply to around 0.62 and 0.50 within the first 10 epochs. This initial learning phase shows successful transfer from the pretrained weights. After epoch 10, improvement continues at a slower rate, with both metrics gradually increasing until they reach approximately 0.75 and 0.6 by epoch 80. The mAP@0.5:0.95 curve follows the same trajectory as mAP@0.5 but consistently lower due to stricter IoU thresholds. The final test performance is 0.7365 mAP@0.5 and 0.6170 mAP@0.5:0.95.

Figure 14: YOLOv11N sign detector mAP@X performance over training epochs

The detector achieves high overall precision of 0.898 but moderate recall of 0.628, resulting in an aggregate mAP@0.5 of 0.777 across all classes. This precision-recall balance indicates conservative prediction behavior where the model requires higher confidence before making detections.

Per-class performance varies substantially (Figure 15). Blind-Spot Mirror (Convex) achieves the highest AP at 0.967. Stop and No Entry perform well at 0.829 and 0.820 respectively. Pedestrian Crossing reaches 0.726, which is notable given that this class contains two distinct rectangular sign forms. The lowest performing classes are Roundabout Ahead at 0.651 and No Through Road (T-Sign) at 0.670. These classes represent only 10% and 8% of the dataset respectively, and the limited training examples likely contribute to their lower detection performance.

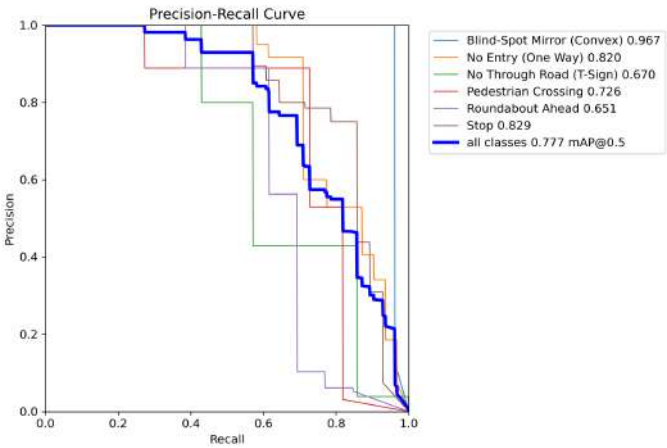


Figure 15: YOLO11N sign detector Precision-Recall Curve for the different sign classes and overall

Figure 16 shows detection examples across different scenarios. Three of the four examples show successful detection. A frontal T-Sign is detected with 0.9 confidence (top-right), a Roundabout Ahead sign viewed from behind receives 1.0 confidence (bottom-left), and a No Entry sign from a side-front angle is correctly identified (bottom-right). The top-left example presents a more challenging case. Two signs are mounted back-to-back on a pole and viewed from an extreme side profile where only a few pixels are visible against a tree background. The detector identifies only the Stop sign at 0.3 confidence while missing the No Entry sign entirely. A street name plate on the pole is incorrectly classified as Pedestrian Crossing. This false positive shows that confusion between rectangular information signs and pedestrian crossing signs persists despite overall improvements in handling the Pedestrian Crossing class. Performance on extreme viewpoints with minimal visual information remains limited.

Figure 16: YOLO11n sign detector detection examples ground truth (left) and prediction (right)

RF-DETR

Figure 17 shows the models' training performance measured by mAP@0.5 and mAP@0.5:0.95 over the total number of training epochs, reflecting the model's learning dynamics and localisation quality.

As seen in all models, both mAP@0.5 and mAP@0.5:0.95 exhibit a sharp increase, particularly within the first 5-10 epochs. The increase from 0.0 to 0.45 and 0.54, respectively, indicates that the RF-DETR Nano model quickly learns to localise traffic signs and is able to distinguish the sign from the background. Following this initial set of epochs, both curves transition into a more gradual improvement and then stabilise. From epoch 15 onwards, mAP@0.5 plateaus at approximately 0.65, while mAP@0.5:0.95 stabilises at around 0.5. This suggests that the model has converged to a stable solution, with further training yielding diminishing returns. The relatively small oscillations around the plateau reflect normal stochastic variation during optimisation and indicate stable training rather than divergence or overfitting. As expected, the persistent gap between mAP@0.5 and mAP@0.5:0.95 shows how the model consistently detects signs correctly at a moderate overlap threshold, however performance drops slightly when stricter localisation accuracy is required. This means that fine-grained bounding box refinement remains more challenging, particularly for smaller background traffic signs where minor spatial errors can significantly reduce IoU at higher thresholds.

Overall, the graph indicates fast convergence, stable optimisation, and good general detection capability, with localisation precision improving more slowly and reaching a lower asymptote. This behaviour is typical for transformer-based detectors trained on relatively small datasets and supports the conclusion that RF-DETR Nano learns sign representations early while progressively refining spatial accuracy over longer training durations.

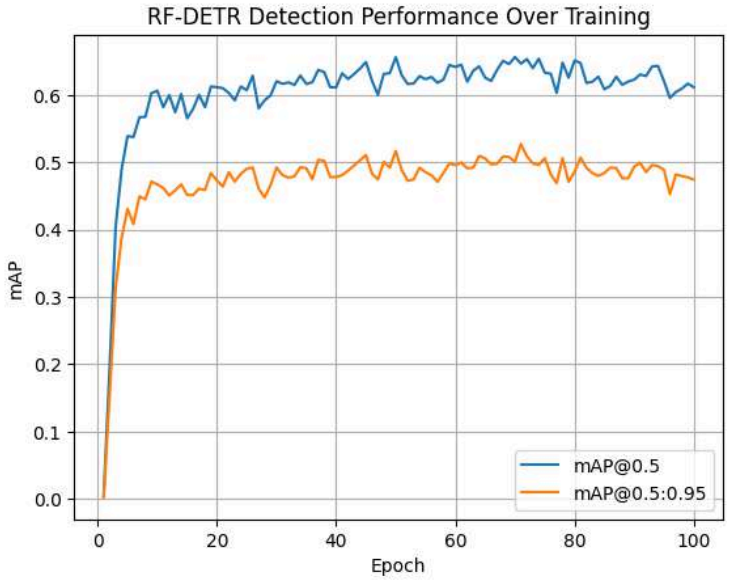
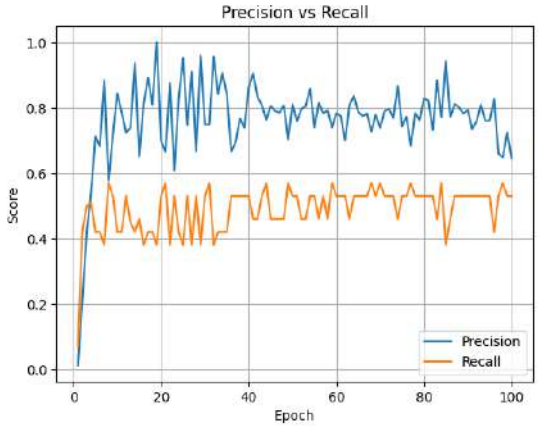


Figure 17: RF-DETRn sign detector mAP@X performance over training epochs

In Figure 18, the precision and recall curves over the full training duration show a rapid initial stabilisation followed by relatively consistent behaviour across later epochs. Precision increases sharply from approximately 0.05 to 0.85 within the first 10 epochs and remains high thereafter, generally fluctuating around the 0.75-0.85 range, indicating that the model quickly learns to suppress false positives and maintain confident predictions. On the other hand, recall also rises across the initial epochs from approximately 0.1 to 0.5, but stabilises at a notably lower level, oscillating around 0.50-0.55. This suggests that, while the detector is conservative and accurate when it predicts an object, it fails to detect a proportion of ground-truth signs, which is a common characteristic in object detection where false negatives are more prevalent than false positives. The limited improvement in recall after early training further indicates that most gains are achieved quickly, with later epochs primarily refining classification confidence rather than expanding coverage.

As seen in Figure 18, this trend is reflected in the final test evaluation of per-class precision and recall results, where most classes exhibit high precision and uniform recall. Visually distinctive signs such as the Blind-Spot Mirror, Stop, No through Road, and No Entry signs achieve very high precision (0.9-1.0), confirming low false-positive rates, while recall remains fixed at the same level (0.55) across all classes. This consistency in recall suggests a global detection threshold or query-selection constraint within the model architecture rather than class-specific failure. In contrast, classes such as Pedestrian Crossing and Roundabout Ahead obtain lower precision (0.58-0.76) due to higher intra-class variability, since both these classes contained two sign shape variations of the same sign type within the dataset (i.e. triangle/square and triangle/circle). Overall, these results show that the model prioritises detection reliability over completeness, producing accurate but conservative predictions across sign categories, meaning the model is more likely to miss detection rather than identifying a false sign.




```
Blind-Spot Mirror (Convex): mAP@50: 0.864, mAP@50:95: 0.783, Precision: 1.000, Recall: 0.550
No Entry (One Way): mAP@50: 0.760, mAP@50:95: 0.595, Precision: 0.905, Recall: 0.550
No Through Road (T-Sign): mAP@50: 0.755, mAP@50:95: 0.600, Precision: 1.000, Recall: 0.550
Pedestrian Crossing: mAP@50: 0.497, mAP@50:95: 0.359, Precision: 0.588, Recall: 0.550
Roundabout Ahead: mAP@50: 0.541, mAP@50:95: 0.461, Precision: 0.769, Recall: 0.550
Stop: mAP@50: 0.766, mAP@50:95: 0.573, Precision: 1.000, Recall: 0.550
all: mAP@50: 0.697, mAP@50:95: 0.562, Precision: 0.877, Recall: 0.550
```

Figure 18: Precision and recall across the total number of training epochs (left) and final testing per-class metrics (right)

Figure 19, shows a sample of different detection examples using RF-DETR Nano. A side viewing angle of a Pedestrian Crossing sign was correctly identified with a confidence of 0.84 (top-left) and a front-side Stop sign opposite a back-side No Entry (One Way) sign where both detected with a confidence of 0.73 and 0.90, respectively (bottom-left). A more complex scenario is presented in the top-right image, which contains multiple sign types observed from different viewing angles. Despite the challenges faced, the model was able to correctly identify the back-facing Blind-Splot Mirror, and 2 background No Entry (One Way) signs with a confidence of 0.96, 0.70, and 0.80, respectively. However, a front-side No Entry (One Way) and back-facing Stop sign weren't detected, indicating that the model can fail to localise overlapping or partially occluded signs when multiple instances compete for attention.

Another challenging example can be seen in the bottom-right corner, where a side-facing Pedestrain Crossing sign was missed entirely. This highlights the model's sensitivity to extreme viewing angles and its reduced accuracy when signs present limited frontal features or reduced visible area. Notably, smaller background signs are detected with lower confidence scores, suggesting increased uncertainty when spatial resolution is limited or when signs occupy a small portion of the image. In general, front-facing signs achieve the highest confidence scores (approximately 0.85-0.95), as their shapes and symbols align closely with the dominant dataset distribution, as seen in the Dataset Section.

Figure 19: RF-DETRn sign detector detection examples ground truth (left) and prediction (right) per example

Faster-RCNN

Figure 20 shows Faster R-CNN detection performance over 100 training epochs using both mAP@0.5 and mAP@0.5:0.95. From the graph it can be seen that both metrics rapidly improve over the first 5-10 epochs, with mAP@0.5 increasing from 0.23 to 0.45 and mAP@0.5:0.95 rising from 0.10 to 0.25, indicating that the detector is learning traffic sign localisation at a steady pace. After the initial 10 epochs, gains started to become more modest, as mAP@0.5 started to stabilise at 20 epoch mark at around 0.48, while mAP@0.5:0.95 edged toward 0.30.

Over the course of the full 100 epochs both curves also showed small oscillations and a few brief dips, suggesting that normal validation is taking place rather than overfitting. Furthermore, at 30 epochs and beyond both curves plateaued with mAP@0.5 having the best score, showing that the model was good at finding signs when the box only needed to be roughly correct, but was less accurate at drawing more precise boxes.

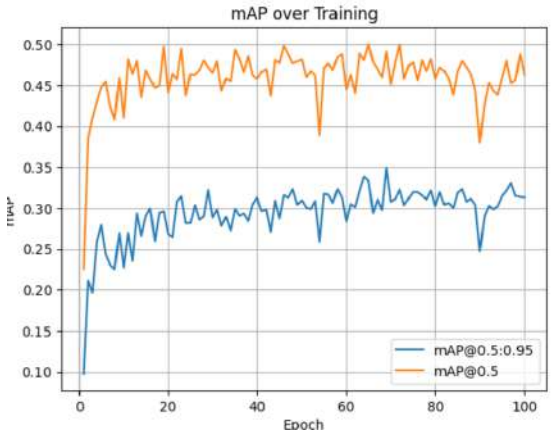


Figure 20: mAP performance over training epochs for Faster R-CNN sign detector

Following mAP testing, precision and recall during training were also evaluated, where figure 21 shows a Precision-Recall curve by class at IoU = 0.5. Overall performance across all classes reached an mAP@0.5 of 0.526, however, some curves indicate substantial variation between sign types, meaning the detector was much more reliable for some classes than others. The strongest classes were Blind-Spot Mirror (0.721) and No through Road (0.645), with their curves staying at high precision for a relatively wide range of recall, suggesting these signs had more consistent visual features and were easier to separate from the background. Stop (0.602) and No-Entry (0.585) also showed moderate performance, with precision declining as recall increased, which is to be expected given that more false positives are introduced. In contrast, Roundabout Ahead (0.394) and Pedestrian Crossing (0.209) performed poorly, likely due to the fact that they were underrepresented in the dataset, or were confused with other similarly shaped signs.

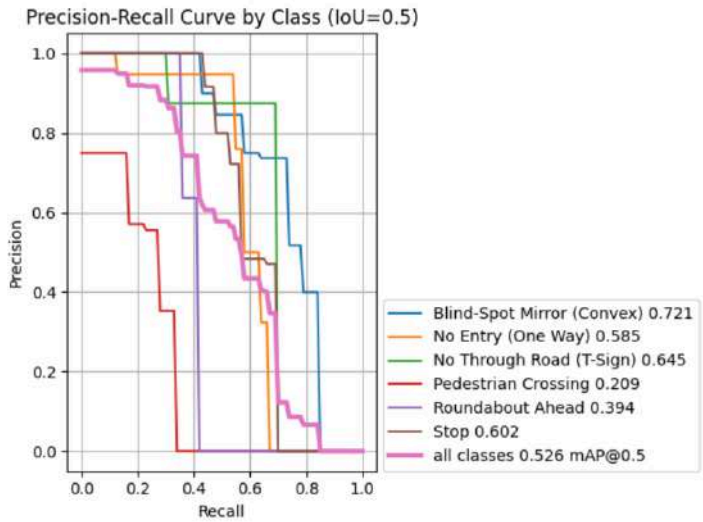


Figure 21: Faster R-CNN sign detector Precision-Recall Curve

In the samples below (Figure 23), the detector correctly identified 5 instances of signs with high confidence, indicating that the model itself learned strong visual cues for each of the identified classes in realistic conditions. As can be seen, the predicted boxes were generally aligned well with the objects, suggesting that the signs were clearly visible and sufficiently in frame, one image in particular produced two boxes, those being No Through Road sign, with the higher confidence sign correctly covering the sign and the other with lower confidence covering the pole itself. This extra box was most likely caused by Faster-RCNN's Region Proposal Network, where a weak candidate proposal survived

post-processing, hence raising the confidence threshold to a higher value would mitigate this issue. Overall, Faster-RCNN displayed very good identification, able to detect various signs in different lighting and backgrounds, whilst also highlighting how different calibration can affect the quantity of predicted boxes.

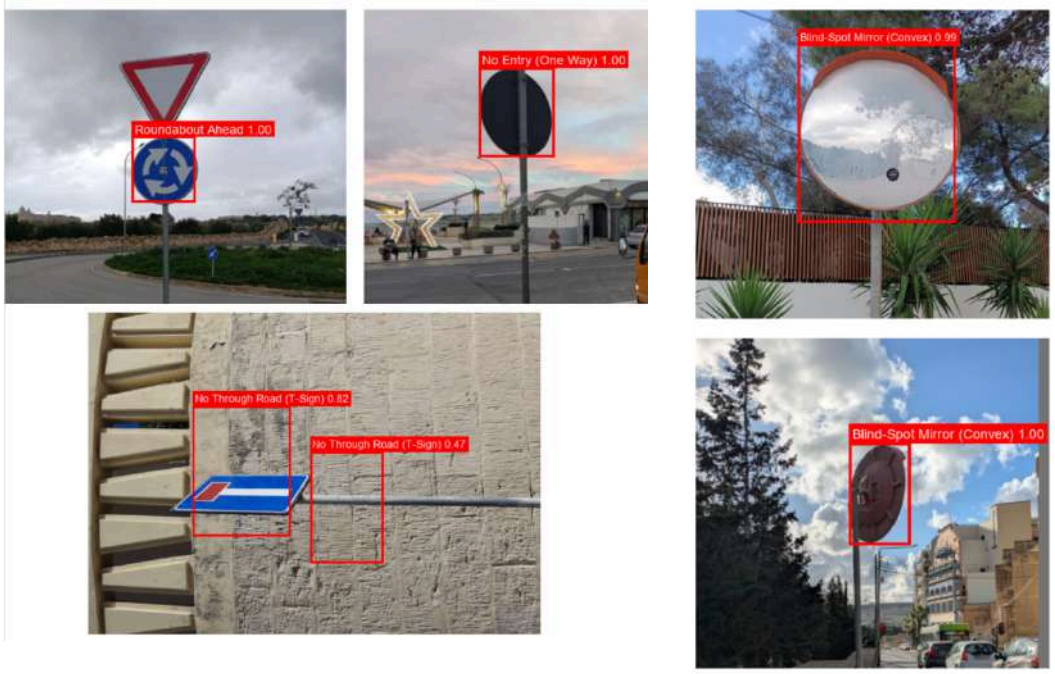


Figure 23: Faster R-CNN sign detector sample images

Evaluation of Attribute Classifiers

ResNet-18 - Angle Classifier

The training dynamics shown in Figure 24 indicates rapid convergence during the early epochs, with both training and validation accuracy increasing quickly, demonstrating effective transfer learning. Validation accuracy stabilises around 0.8, while training accuracy gradually increases to approximately 0.95. This divergence suggests mild but controlled overfitting, with no evidence of validation performance degradation. The plateau in validation accuracy indicates that the model has converged, and extending training beyond 25 epochs would likely provide limited benefit.

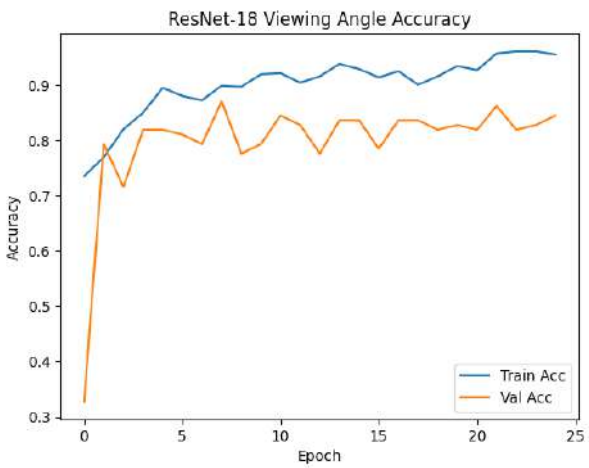
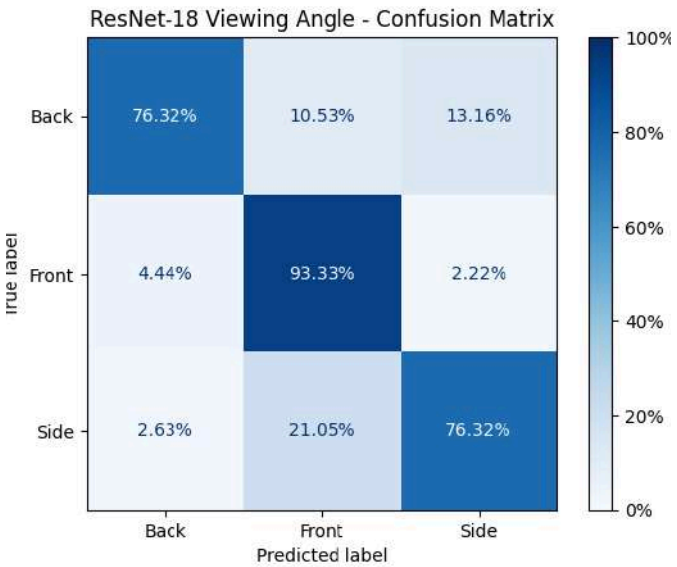


Figure 24: ResNet-18 angle classification, train and validation accuracy over epochs

The percentage-based confusion matrix and classification report (Figure 25) provides a consistent and complementary evaluation of model performance. The classifier achieves a test accuracy of 0.826, with macro and weighted F1-scores of 0.824 and 0.825 respectively, indicating balanced performance across the three viewing-angle classes.

The Front class performs strongest, with 93.33% of frontal views correctly classified, corresponding directly to its recall of 0.933. Misclassification of frontal views is minimal, while its lower precision (0.778) reflects the tendency for some Back and Side samples to be predicted as Front. The Back and Side classes exhibit similar behaviour, each achieving a 76.32% correct classification rate. Back views are confused with Front (10.53%) and Side (13.16%), while Side views are most frequently misclassified as Front (21.05%). These structured error patterns explain the higher precision scores for Back (0.906) and Side (0.829) relative to Front, indicating more conservative and reliable predictions for these classes.

Overall, misclassifications primarily occur between adjacent viewpoints (Back–Side and Side–Front), reflecting the inherent visual ambiguity of side-oriented perspectives rather than random errors. The alignment between the confusion matrix and precision–recall statistics confirms that the model has learned meaningful orientation-dependent features.



	precision	recall	f1-score	support
Back	0.906	0.763	0.829	38
Front	0.778	0.933	0.848	45
Side	0.829	0.763	0.795	38
accuracy			0.826	121
macro avg	0.838	0.820	0.824	121
weighted avg	0.834	0.826	0.825	121

Figure 25: ResNet-18 angle classification, confusion matrix (left) and classification report (right)

The qualitative examples in Figure 26 further supports this analysis. Correct predictions are observed when clear geometric cues such as edges and surface orientation are visible. The misclassified example shows a Side view predicted as Front under heavy blur and reduced perspective information, reinforcing that remaining errors arise from ambiguous or degraded visual conditions rather than poor model generalisation.

Figure 26: ResNet-18 angle classification, Ground truth (GT) vs Prediction (pred)

ResNet-34 - Sign Shape Classifier

Training progressed rapidly in the initial phase (Figure 27), with accuracy jumping from 55% to around 75-80% within the first three epochs. This sharp improvement indicates successful transfer of ImageNet-learned features to the sign shape task. The model reached peak validation performance at epoch 7 (~85% accuracy), and this checkpoint was saved as the final model. A clear overfitting pattern emerged after epoch 7. Training accuracy continued climbing to approximately 88%, while validation accuracy oscillated erratically between 58% and 80%. Early stopping intervened at epoch 12, preventing further deterioration. The unstable validation curve after epoch 7 indicates the model had exhausted its generalization capacity on this dataset.

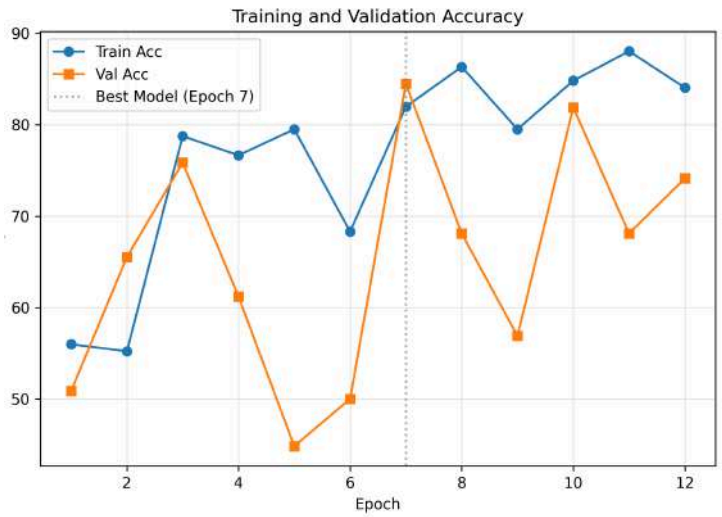


Figure 27: ResNet-34 sign shape classification, train and validation accuracy over epochs

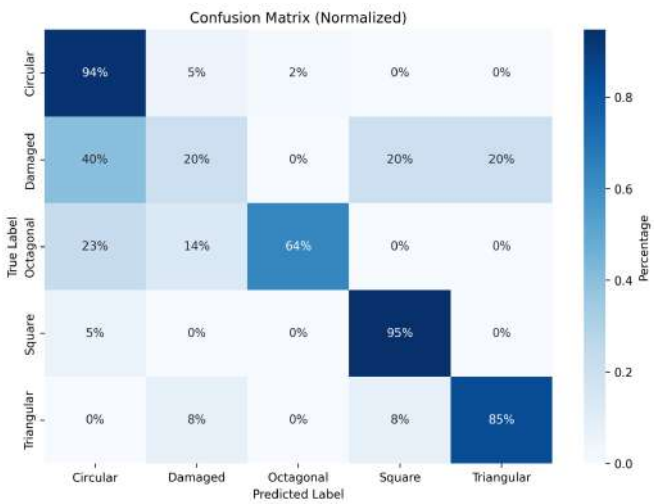
On the test set (Figure 28), the model achieves an overall accuracy of 84.3%. The weighted F1-score of 0.848 significantly exceeds the macro F1-score of 0.724, indicating uneven performance across shape categories. Circular signs dominate the test set (62 of 121 samples) and achieve 93.55% recall,

indicating that nearly all circular signs are correctly identified. However, the precision is 87.88%, indicating that when the model is uncertain about other shapes, it tends to default to predicting Circular.

Square signs show the strongest overall metrics, with 94.74% recall and 90.00% precision. Their distinctive rectangular geometry appears particularly easy for the model to recognize. Triangular signs also perform well (84.62% recall, 91.67% precision) despite limited test samples (13 total), suggesting their angular features are sufficiently discriminative.

Octagonal signs achieve 93.33% precision but only 63.64% recall. When the model predicts Octagonal, it is usually correct. However, it misses many actual octagonal signs, classifying 23% as Circular and 14% as Damaged. This pattern likely results from visual similarity between octagons and circles, particularly when stop signs have worn or rounded corners.

The Damaged class performs poorly with 20% recall and 12.50% precision across just five test samples. Predictions are distributed across all other categories, with 40% classified as Circular, 20% as Square, and 20% as Triangular. This poor performance results from a conceptual problem with the class definition, as “Damaged” describes a condition rather than a geometric shape. Unlike the four geometric categories, it lacks consistent visual features for the model to learn. The class conflates physical damage with shape identity, making it difficult to classify reliably.




```
Classification Report:
              precision    recall  f1-score   support

    Circular     0.8788    0.9355    0.9062        62
     Damaged     0.1250    0.2000    0.1538         5
   Octagonal     0.9333    0.6364    0.7568        22
      Square     0.9000    0.9474    0.9231        19
  Triangular     0.9167    0.8462    0.8800        13

    accuracy                         0.8430       121
   macro avg     0.7508    0.7131    0.7240       121
weighted avg     0.8650    0.8430    0.8478       121
```

Figure 28: ResNet-34 sign shape classification, confusion matrix (left) and classification report (right)

Visual examples (Figure 29) support these findings. Correct predictions occur when signs are unobstructed with clear geometric features (top row), while the misclassifications reveal specific failure modes. One Octagonal sign was predicted as Circular, but the cropped region shows significant occlusion by a foreground circular sign. This represents an inherited detection error rather than a pure classification failure. A severely damaged Octogonal sign labeled as Damaged was predicted as Square, showing how the labeling conflates damage state with geometric shape. A flat printed circular sign was predicted as Damaged, possibly suggesting that the model has learned to associate visual anomalies such as unusual texture or missing depth cues with the Damaged label rather than actual geometric damage.

Figure 29: ResNet-34 sign shape classification, Ground truth (GT) vs Prediction (pred)

The ResNet-34 classifier performs well when geometric features are clear and signs are properly framed. Most errors result from either ambiguous inputs due to occlusion or severe viewpoint angles, or from the ill-defined Damaged category. The confusion patterns between Octagonal and Circular shapes and the use of Damaged as an anomaly flag indicate that the model learns genuine geometric relationships but struggles when input crops are compromised or when the labeling scheme conflates shape with condition.

EfficientNet-B0 - Mounting Type Classifier

Figure 30 shows training and validation accuracy of the EfficientNet-B0 mounting-type classifier over 5 epochs. Here it can be seen that training accuracy increased steadily from approximately 0.86 at epoch 1 to 1.00 by epoch 5, indicating that model fit the training set extremely quickly, where in contrast the validation accuracy improved only modestly, rising from about 0.49 to 0.59 across the exact same period.

Furthermore, the widening gap between the training and validation curves themselves suggest that overfitting is present, meaning the network learned patterns very well on training images but not on validation ones. Extending training to additional epochs did not improve validation accuracy, indicating that the limitation was not optimization but generalisation. One likely contributor to this was class imbalance, as the dataset contained a high proportion of pole-mounted signs, which may have biased the classifier toward the majority class and reduced performance on the wall-mounted sign type.

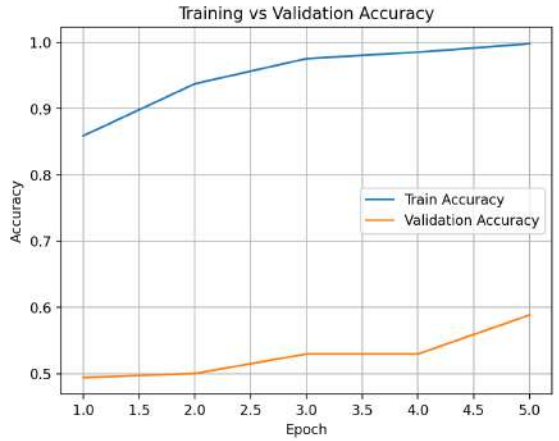
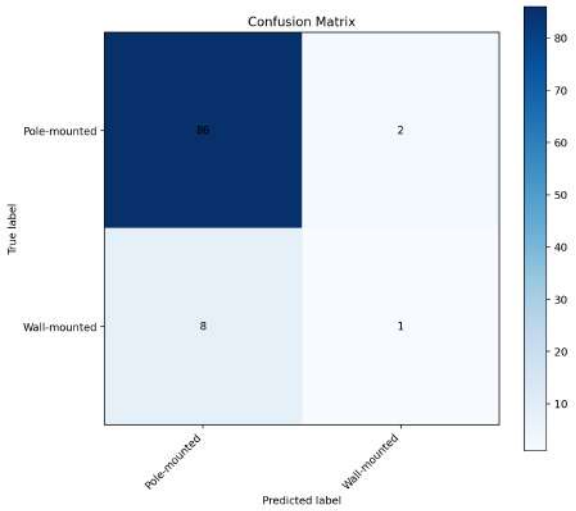


Figure 30: EfficientNet-B0 mounting type classification (Train Vs Val)

The confusion matrix seen in Figure 31 further shows strong class imbalance and clear bias, where the test set contained 88 pole-mounted samples, but only 9 wall-mounted, thus overall accuracy was dominated by how well the model handled the formers' signs. As expected, pole-mounted performance was high with 86/88 being classified correctly, also matching with the classification report achieving a precision of 0.915, recall of 0.977 and F1 score of 0.945. However, the direct opposite for wall-mounted was also true, only having 1/9 signs being correctly identified, with a low recall of 0.111 and F1 score of 0.167.

Overall, the macro averages show the following results; precision = 0.624 recall = 0.544, F1 score =0.556, highlighting that performance drops substantially once both classes are weighted equally. Hence, this supports the earlier conclusion of overfitting, as the classifier learned the dominant pole-mounted pattern but evidently struggled to generalise for the underrepresented wall-mounted class.




```
Classification Report (per class)
Class                          Precision     Recall         F1    Support
--------------------------------------------------------------------------
Pole-mounted                      0.9149     0.9773     0.9451         88
Wall-mounted                      0.3333     0.1111     0.1667          9
--------------------------------------------------------------------------
MACRO AVG                         0.6241     0.5442     0.5559         97
```

Figure 31: EfficientNet-B0 mounting type classification, confusion matrix (left) and classification report (right).

Sample images which can be seen in Figure 32 also supports the analysis given thus far for this classifier, as images with ground-truth wall-mounted signs were frequently predicted as pole-mounted signs, which was consistent with the results seen in the confusion matrix. However, overall the model correctly identified pole-mounted signs, particularly when the crop preserved contextual cues such as the pole itself, or the sign's side profile which often revealed a visible attachment at the back. Aside from the need for more wall-mounted signs in the dataset, a future improvement would be to use a more

generous cropping strategy, specifically balancing the need for features such as walls or brackets that connect to the sign, whilst also minimising as much background noise as possible.

Figure 32: EfficientNet-B0 mounting type classification, Ground truth (GT) vs Prediction (pred)

EfficientNet-B2 - Sign Condition Classifier

Figure 33 illustrates the training and validation accuracy over the total number epochs, indicating a generalisation gap between training and validation. Training accuracy increases rapidly over the total number of epochs, from approximately 0.61 to nearly 0.99, indicating that EfficientNet-B2, together with the AdamW optimiser, effectively fits the training data. In contrast, validation accuracy remains in the range of 0.68-0.71, peaking at approximating 0.71 by epoch three and failing to improve thereafter. Despite using a small number of epochs and early stopping to mitigate overfitting as much as possible, this divergence reflects how the model is overfitting during training. This means the model memorises the training samples but may struggles to generalise to unseen data, which is a common outcome when fine-tuning a high-capacity backbone on a relatively small dataset.

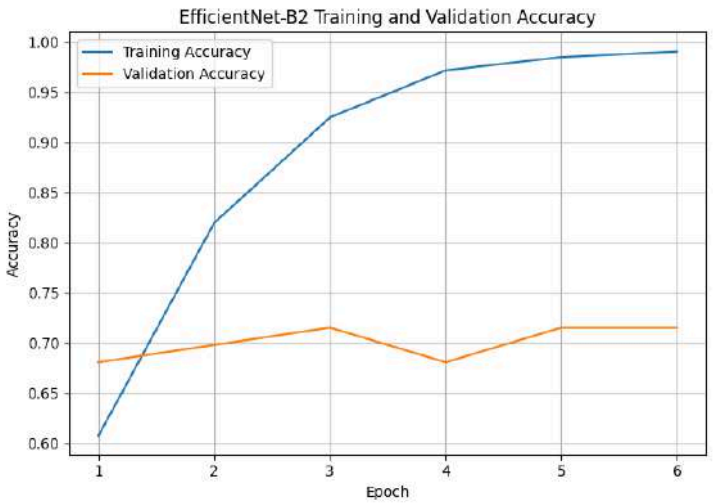
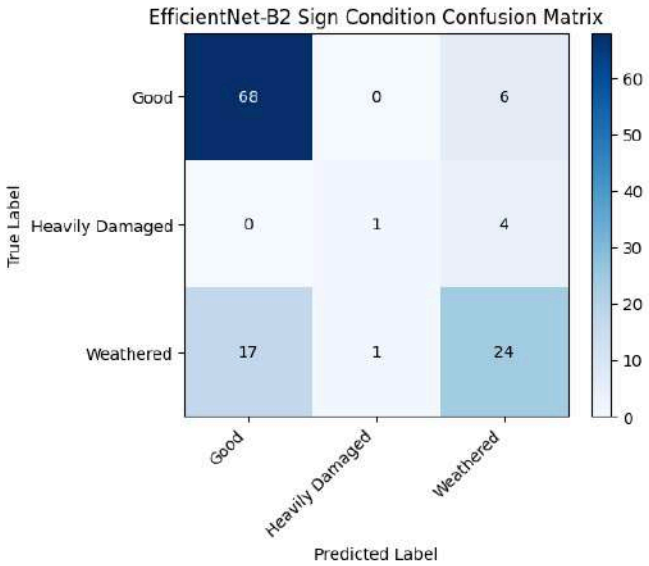


Figure 33: EfficientNet-B2 sign condition classification, train and validation accuracy over epochs

To perform an evaluation of the test performance of EffientNet-B2, a confusion matrix and classification report where used, seen in Figure 34. The confusion matrix for the EfficientNet-B2 sign condition

classifier reveals a strong bias toward the majority good class, reflecting the underlying imbalance in the dataset, where 66% of samples are labelled as good, 29% as weathered, and only 5% as heavily damaged. The model correctly classifies 68 good-condition signs but misclassifies 6 as weathered, indicating high separability between good and heavily damaged signs but confusion with intermediate degradation states. For the heavily damaged class, only 1 instance is correctly identified, while 4 samples are misclassified as weathered, indicating the difficulty of learning robust features for this class due to its underrepresentation and feature similarity to the weathered signs. The weathered class, shows the highest confusion, with 24 correct predictions, 17 misclassified as good and 1 as heavily damaged, suggesting substantial visual overlap between lightly degraded and intact signs. Overall, these results indicate that the classifier prioritises majority-class patterns and struggles to discriminate subtle condition differences, particularly when class boundaries are gradual and the dataset imbalance, limiting the model's exposure to weathered and heavily damaged sign condition patterns.

The classification report indicates that the EfficientNet-B2 model performs well on the majority Good class, achieving a precision of 0.8, high recall (0.92) and a good F1-score (0.86), which indicates reliable identification of signs that fall under this class. Performance on the Weathered class is moderate, with balanced but lower precision of 0.7, and precision and recall (0.71 and 0.57), suggesting confusion with visually similar conditions, as seen in the confusion matrix. As discussed in the sign shape classification section, the Heavily Damaged class exhibits poor recall (0.20) and a low F1-score (0.29), with a precision of 0.5, highlighting the model's limited ability to detect severe damage due to class imbalance and insufficient representative samples. Hence, the overall accuracy of 0.77 is primarily driven by the dominant classes, masking weak performance on minority categories.



	precision	recall	f1-score	support
Good	0.8000	0.9189	0.8553	74
Heavily Damaged	0.5000	0.2000	0.2857	5
Weathered	0.7059	0.5714	0.6316	42
accuracy			0.7686	121
macro avg	0.6686	0.5634	0.5909	121
weighted avg	0.7549	0.7686	0.7541	121

Figure 34: EfficientNet-B2 sign condition classification, confusion matrix (left) and classification report (right)

Figure 35, shows a sample of different sign condition classification examples using EfficientNet-B2. The bottom row and top-left and middle images demonstrate good sign condition classification of both Good and Weathered conditions of different sign types, including Roundabout Ahead, No Through Road (T-Sign), and No Entry (One Way), indicating that the model effectively captures distinct surface-level features. In contrast, the top-right example illustrates a misclassification where a Heavily Damaged sign is predicted as Weathered, reflecting the model's difficulty in distinguishing severe degradation from moderate weathering when these features overlap. This behaviour aligns with the quantitative results obtained earlier, where heavily damaged signs are underrepresented and exhibit low precision and recall. Overall, the EfficientNet-B2 classifier demonstrates good performance for dominant sign conditions but remains limited in reliably when classifying severely damaged signs, highlighting the

impact of class imbalance and the need for more representative data or targeted regularisation strategies.

Figure 35: EfficientNet-B2 sign condition classification, Ground truth (GT) vs Prediction (Pred)

Final Group Analysis

Model Name	mAP@0.5	mAP@0.5:0.95	Precision	Recall
YOLO11n	0.74	0.62	0.90	0.63
YOLOv8n	0.72	0.58	0.85	0.64
RF-DETR Nano	0.69	0.56	0.87	0.55
FasterRCNN	0.53	0.36	0.53	0.54

Table 2: Performance summary of detector models.

Table 2 displays the mAP@0.5 and mAP@0.5:0.95 values for all the object detectors, using the COCO-based_COCO dataset, including YOLOv11n, YOLOv8n, RF-DETR Nano, and FasterRCNN MobileNetv3. These results provide insight into both the detection capability and localisation precision.

YOLO11n achieves the highest mAP@0.5 and mAP@0.5:0.95 with values 0.74 and 0.62, followed by YOLOv8n with values 0.72 and 0.58, and RF-DETR Nano with values 0.69 and 0.56, respectively. This indicates that these three models are effective at detecting traffic signs and maintaining reasonably accurate localisation. Their performance can be attributed to YOLO's single-stage design, optimised feature pyramids, and transformer-based attention mechanisms, which is beneficial when using well-defined objects such as traffic signs and detecting smaller background signs. However, RF-DETRs' lower mAP@0.5 and mAP@0.5:0.95 values compared to YOLO models indicates that it misses slightly more detections overall. This behavior aligns with transformer-based detectors, which often prioritise global context and localisation quality especially when trained from scratch on a limited dataset. Faster R-CNN obtains the lowest performance, with mAP@0.5 and mAP@0.5:0.95 corresponding to values 0.53 and 0.36, respectively, indicating imprecise bounding box refinement and weaker localisation accuracy. While two-stage detectors are traditionally strong in accuracy, it's performance here is due to dataset size and the presence of small background signs, where region proposal mechanisms can struggle.

Overall, these results show that YOLO-based and transformer-based models offer the best balance between detection accuracy and localisation precision for this traffic sign dataset, making them well-suited even when the dataset size is limited. RF-DETR Nano demonstrates good localisation consistency but would benefit from the use of pretrained weights, since for this assignment the model was trained on the dataset we created. Additionally, since transformer-based detectors require large amounts of data when being trained, the model would benefit from a larger dataset size. Faster R-CNN achieves the lowest performance in this setting, underscoring the challenges of applying heavier two-stage detectors to small-object detection tasks with limited training data. Moreover, the moderate drop from mAP@0.5 to mAP@0.5:0.95 is present in all models, suggesting good but not perfect bounding box tightness when using stricter IoU thresholds across all models, as seen in Figure 36.

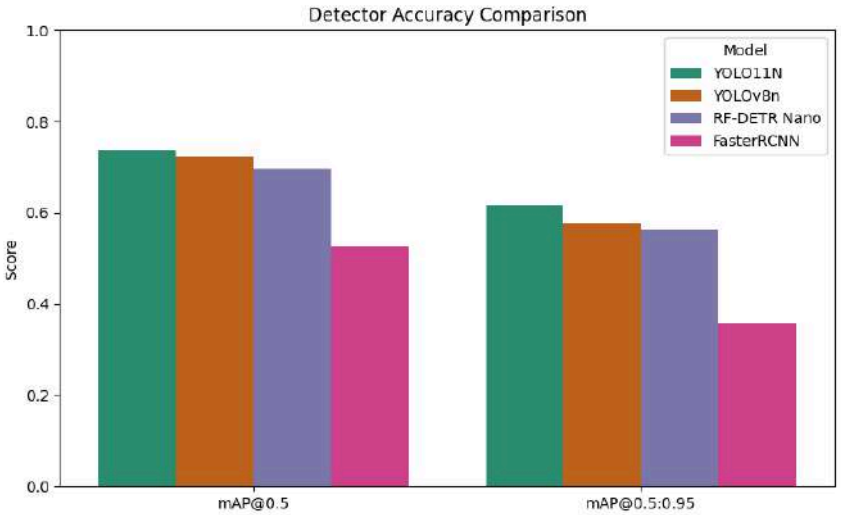


Figure 36: mAP@0.5 and mAP@0.5:0.95 results per model

Figure 37 illustrates all the results obtained for all the object detectors with respect to precision and recall. This plot indicates how many of the detected signs are actually correct as well as how many of the true signs were successfully detected.

YOLO11n achieves the highest precision among all models with a value of 0.90, indicating that most detections it makes are correct and only a few false positives were obtained. However, its recall is relatively lower, meaning it misses a number of true signs, thus more false negatives. This suggests that YOLO11n is conservative, meaning it prefers to be confident in detections rather than detecting false positives. As a result, it's suitable when false alarms are costly, but it may not be ideal when full coverage is required. Similarly, RF-DETR Nano demonstrates good precision at 0.87, indicating reliable detections with a modest increase in false positives compared to YOLO11n. However, its recall of 0.55 shows that nearly half of the true signs remain undetected. This suggests that RF-DETR Nano also adopts a conservative detection strategy, where detections are generally accurate when produced but coverage is limited. The reduced recall is likely influenced by the model's difficulty in consistently identifying smaller or less salient background signs, reflecting the trade-off between detection confidence and completeness inherent in its architecture.

YOLOv8n shows a good balance between precision and recall, with slightly lower precision than RF-DETR but marginally higher recall than YOLOv11n. This indicates that it detects more true signs than YOLO11n while still maintaining a high rate of true positives. As a result, YOLOv8n is less conservative than RF-DETR Nano and YOLO11n, improving coverage at the cost of a small increase in false positives. Overall, YOLOv8n, RF-DETR Nano, and YOLOv11n tend to prioritise confident detections over exhaustive coverage, indicating a conservative detection behaviour. FasterRCNN shows the weakest performance in precision and recall, with a notable drop in compared to the YOLO and transformer-based models. A precision close to 0.5 means the model is almost equally likely to be wrong as right when it predicts a sign, and recall near 0.54 indicates it detects only slightly more than half of the true signs. This suggests some issues with both detection quality and coverage, potentially due to the model's sensitivity to small background traffic signs, making it the least reliable model for this task under the current conditions.

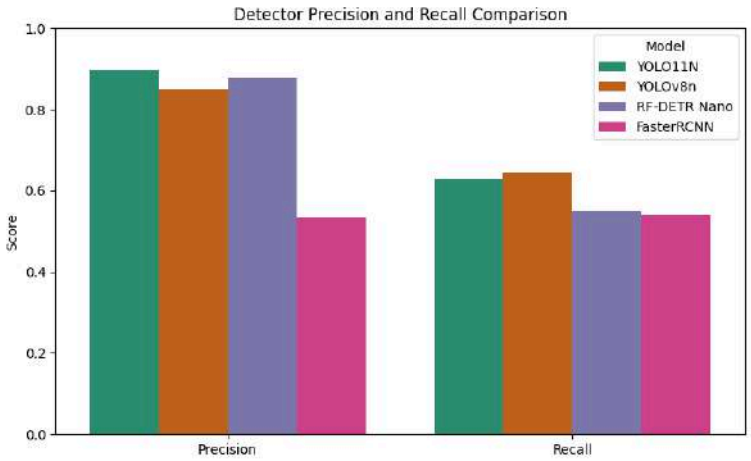


Figure 37: Precision and Recall results per model

Model Name	Test Accuracy	F1-Score	Precision	Recall
ResNet34	0.84	0.85	0.86	0.84
ResNet18	0.83	0.82	0.84	0.82
EfficientNet-B2	0.77	0.75	0.75	0.77
EfficientNet-B0	0.54	0.56	0.62	0.54

Table 3: Performance summary of classifier models.

Table 3 lists the test accuracy and F1-score for all attribute classifiers, including EfficientNet-B0, EfficientNet-B2, ResNet18, and ResNet34 for mounting type, sign condition, viewing angle, and sign shape, respectively. Since we're comparing different models over different sign attributes this wouldn't make a fair comparison, hence this comparison is being made to get a better idea of which attribute classification tasks are more difficult. Each model was selected to best suit the characteristics of its respective attribute, allowing the comparison to highlight which sign attributes are more challenging to classify rather than to rank model performance across architectures.

EfficientNet-B0 shows the weakest performance among all attribute classifiers, indicating difficulty in reliably distinguishing mounting types (i.e. pole or wall mounted). The low accuracy and F1-score suggest both frequent misclassifications and inconsistent class-wise performance, likely due to the limited representational capacity of the smaller EfficientNet models and cropping which might have affected pole visibility. On the other hand, EfficientNet-B2 achieves solid performance on sign condition classification, indicating effective learning of texture, wear, and surface degradation features. The model managed to obtain an accuracy of 0.77 and an F1-score of 0.75, making it well-suited for capturing visual condition attributes (i.e. Good, Heavily Damaged, and Weathered) that depend on fine-grained appearance features. Nonetheless, the model still struggles when classifying heavily damaged signs which are often misclassified as weathered due to the underrepresentation of heavily damaged signs within the dataset.

ResNet18 performs well for viewing angle classification, demonstrating good generalisation and consistent predictions across classes (i.e. Side, Back, and Front). The close alignment between accuracy and F1-score indicates reliable detection across different viewpoints, suggesting that the residual architecture effectively captures geometric and orientation-related features. Overall, ResNet34 achieves the highest performance, reflecting good capability in classifying sign shapes (i.e. Triangular, Square, Octagonal, Circular, Damaged). The high F1-score of 0.85 indicates strong precision and recall,

implying that the deeper ResNet architecture effectively models structural features that are critical for shape recognition. Similar to sign condition classification, ResNet-34 also exhibits difficulty in recognising the damaged category, which is frequently misclassified as square.

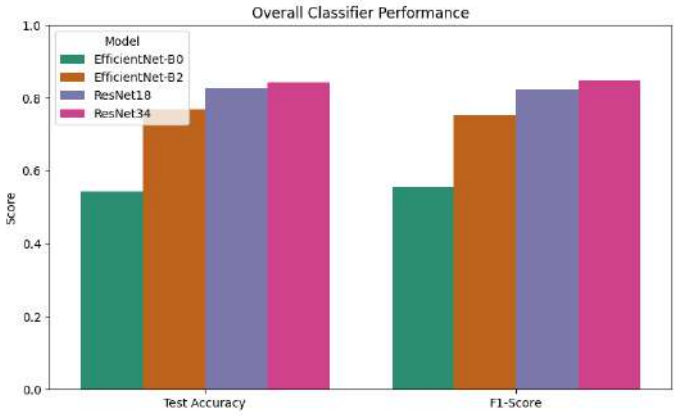


Figure 38: Test accuracy and F1-Score results per model

Figure 39 illustrates all the results obtained for all the classifiers with respect to precision and recall. EfficientNet-B0 exhibits the weakest recall, indicating that a substantial portion of true instances are missed due to a high number of false negatives. The lower precision further suggests increased false positives, reflecting limited discriminative capacity for complex mounting type scenarios. This implies that EfficientNet-B0 struggles with mounting type classification, possibly due to its lower capacity in comparison to larger EfficientNet models and cropping which might have limited pole visibility. On the other hand, EfficientNet-B2 shows moderate and well-balanced precision and recall, obtaining values 0.75 and 0.77 respectively, indicating reasonable classification reliability but with more confusion between classes than the ResNet models. The slightly higher recall than precision suggests the model is more inclusive in its predictions, capturing most true instances at the cost of increased false positives. This behaviour is consistent with appearance-based attribute classification.

ResNet18 also performs consistently well, with only a marginal drop compared to ResNet34. A precision of 0.84 indicates few false positives, and a recall of 0.82 suggests good coverage of true viewing angles. This implies that ResNet18 offers an effective trade-off between model complexity and classification reliability, making it a good lightweight alternative to deeper architectures. ResNet34 achieves the strongest performance in both precision and recall with values 0.86 and 0.84, respectively, indicating highly reliable and well-balanced predictions. The high precision shows that most predicted classes are correct, while the recall demonstrates that the model successfully captures the majority of true sign shapes. This balance suggests strong generalisation across sign shape classes, particularly for structurally distinct attributes, given the clear and easily distinguishable edge and shape features.

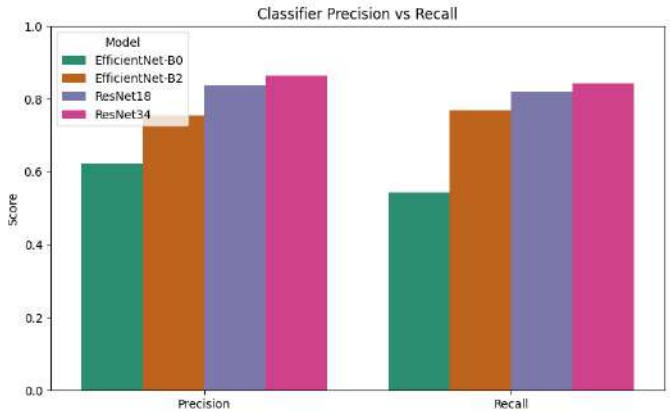


Figure 39: Precision and Recall results per model

In conclusion, YOLO11n achieved the highest object detection performance, followed by YOLOv8n, RF-DETR Nano, and Faster R-CNN, reflecting a clear trade-off between accuracy and model complexity. For attribute classification, ResNet-34, ResNet-18, EfficientNet-B2, and EfficientNet-B0 proved most effective for sign shape, viewing angle, sign condition, and mounting type, respectively, highlighting the need to select architectures according to the specific requirements of each classification task.

References and List of Resources Used

[1] Carion, Nicolas, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. 2020. *End-to-End Object Detection with Transformers*. Edited by Andrea Vedaldi, Horst Bischof, Thomas Brox, and Jan-Michael Frahm. Vol. 12346. N.p.: Springer International Publishing. 10.1007/978-3-030-58452-8_13.

[2] Jocher, Glenn, and Jing Qiu. 2024. Ultralytics YOLO11. https://docs.ultralytics.com/models/yolo11/#citations-and-acknowledgments.

[3] Oommen, Prem. 2020. “ResNets – Residual Blocks & Deep Residual Learning.” https://towardsdatascience.com/resnets-residual-blocks-deep-residual-learning-a231a0ee73d2/.

[4] Redmon, Joseph. 2025. “YOLO Object Detection Explained: Real-Time Vision Tasks.” Roboflow Blog. https://blog.roboflow.com/yolo-object-detection/.

[5] Ren, Shaoqing, Kaiming He, Ross Girshick, and Jian Sun. 2017. “Faster R-CNN: Towards real-time object detection with region proposal networks.” *IEEE Transactions on Pattern Analysis and Machine Intelligence* 39 (6): 1137-1149. https://doi.org/10.1109/TPAMI.2016.2577031.

[6] Robinson, Isaac, Peter Robicheaux, Matvei Popov, Deva Ramanan, and Neehar Peri. 2025. “RF-DETR: Neural Architecture Search for Real-Time Detection Transformers.” (Nov). https://arxiv.org/abs/2511.09554.

[7] Tan, Mingxing, and Quoc V. Le. 2019. “EfficientNet: Rethinking model scaling for convolutional neural networks.”

[8] Wang, Ao, Hui Chen, Lihao Liu, Kai Chen, Zijia Lin, Jungong Han, and Guiguang Ding. n.d. “YOLOv10: Real-Time End-to-End Object Detection.” https://arxiv.org/abs/2405.14458.

[9] Wang, Chien-Yao, Alexey Bochkovskiy, and Hong-Yuan M. Liao. 2022. “YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors.” https://arxiv.org/abs/2207.02696.

[10] Wang, Chien-Yao, I-Hau Yeh, and Hong-Yuan M. Liao. 2024. “YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information.” (Feb). https://arxiv.org/abs/2402.13616.

[11] Wu, Xiongwei, Doyen Sahoo, and Steven C. Hoi. 2020. “Recent advances in deep learning for object detection.” *Neurocomputing* 396:39-64. 10.1016/j.neucom.2020.01.085.

[12] YOLO. n.d. Ultralytics YOLO Docs: Home. Accessed January 8, 2026. https://docs.ultralytics.com/.

Generative AI Usage

Introduction

Throughout the project, Gemini 3, Claude Sonnet 4.5 and ChatGPT's GPT-5.2 were the main AI models that were used. Claude was mainly used for the initial code planning phase because of its robustness in code design and logic structure. ChatGPT's GPT-5.2 was then used for the implementation phase for its efficient, clean code generation. Gemini was used when explanations were needed for complex concepts and outputs because of its high usage limits.

Ethical Considerations

The use of generative AI in this project raises several important considerations. Since large language models are trained on internet text, they may contain biased perspectives, outdated information or underrepresented viewpoints in computer vision research. This bias can take many forms. The AI might favor certain algorithmic approaches over others based on their popularity in online discussions or history rather than their technical rigor. This may potentially overlook newer or less commonly discussed methods.

While this project involved publicly available computer vision algorithms, the use of generative AI means that the prompts and code segments shared with the system are stored by the providers. Although the technical content of this project does not contain any sensitive information, it is still important to be mindful about what information is shared with AI systems.

While AI provided explanations of existing concepts and helped debug implementation issues, the challenge lies in distinguishing between AI-assisted learning and AI-generated content. This project maintained academic integrity by using AI primarily as an educational and debugging tool rather than as a code generator. This is done to make sure that any analysis or conclusions are from group member's understanding and reasoning.

Methodology

The generative AI models were accessed through their respective web interfaces. Generated code outputs were reviewed and modified as needed before being implemented into the source code.

Prompts and Responses

Daniel Borg Finn

Training Time Improvement

Prompt:
My current object detector is taking too long to train per epoch, how can I make this more efficient?

Reflection:
This helped in researching different versions of a model that offer much better performance when it comes to training time.

Bounding Box Accuracy

Prompt:
My mAP@0.5 is decent but mAP@0.5:0.95 is noticeably lower. What does this indicate about localisation quality?

Reflection:
This clarified they key difference between the two metrics, and what it said about the models performance in terms of stricter metrics.

Mounting-type classifier design

Prompt:
For mounting-type classification, before training should I crop the images, and how will this effect results?

Reflection:
The AI gave a clear answer in how reducing background noise can positively affect the robustness of the model, leading to a greater understanding in how real-life models are designed.

Daniela Curmi

Dataset Preparation

Prompt:

Prepare the COCO-based_COCO Dataset for the RF-DETR Nano model by creating three folders called train, test and valid. My current folder structure is divided into 2 folders, including an annotation folder and an images folder. The annotation folder contains 3 json files corresponding to each dataset split and my images folder contains three folder with images corresponding to each dataset split. Convert the 3 annotation files into COCO format and a subcategory for all categories called 'sign', if missing.

Reflection:
The prompt encouraged a more structured and systematic approach to dataset preparation, improving my understanding of model training requirements. It was also important for directory organization and annotation consistency when adapting data for transformer-based detection models such as RF-DETR Nano.

Drawing Bounding Boxes and Ground Truth Images

Prompt:

Can you create a method to draw bounding boxes, including label name and confidence score (if it's a prediction)? and can you also provide a method to get the ground truth images and their annotations so that I can compare the predicted bounding boxes to ground truth?

Reflection:
This prompt provided a clearer understanding of model evaluation by emphasising the importance of visual comparison between predicted and ground truth bounding boxes, providing a good qualitative analysis of RF-DETR object detection performance.

Plotting Results for RF-DETR

Prompt:

Using the file called log.txt which contains results for all 100 epochs during training, can you plot mAP@0.5 and maP@0.5:0.95 over training and the precision and recall values over training?

Reflection:
This prompt helped support my model evaluation and my ability to interpret training dynamics by using the visualisation of performance metrics over time, thereby improving my understanding of how mAP, precision, and recall evolve across epochs and inform model convergence and generalisation.

Matthew Pirotta

Evaluation metrics and result interpretation

Prompt:

How should mAP@0.5 and mAP@0.5:0.95 be interpreted when comparing object detectors, and what limitations should I mention in an academic report?

Reflection:
This helped ensure correct interpretation of results, improving the quality of evaluation and discussion.

Debugging and code-level assistance

Prompt:

My PyTorch classification model always reports zero accuracy even though predictions are generated. What are common causes, and how should I verify my labels and predictions?

Reflection:
AI was used as a debugging assistant, accelerating problem diagnosis. The actual fix required converting python lists to tensors.

Report writing

Prompt:

Can you help me rephrase this paragraph to be concise, academic, and specific to a computer vision project?

Reflection:
The AI assisted with clarity and structure, not content invention. The reflection remained personal and project-specific.

Matthew Vella

Memory management during training

Prompt:

My YOLO training crashes with 'OpenCV Error: Insufficient memory' when loading images, even though I have an RTX 4060 with 8GB VRAM. I'm using batch=16, workers=8, and imgsz=640. What's causing the RAM issue and how should I configure training for my system?

Reflection:
The AI identified that workers=8 was launching 8 processes that simultaneously loaded images into system RAM, not GPU memory. Setting workers=0 and reducing batch size resolved the crash and allowed training to complete successfully.

Image orientation issues

Prompt:

I'm extracting crops from YOLO bounding boxes, and some crops show the wrong parts of the image. When I view the original images, the bounding boxes look correct, but when Python loads and crops them, the same coordinates are off. What could be causing this?

Reflection:
The AI identified that smartphone images store rotation in EXIF metadata. Image viewers auto-apply this, but Python's Image.open() reads raw unrotated pixels. Adding ImageOps.exif_transpose() before cropping fixed the misalignment by applying the rotation first.

Handling class imbalance

Prompt:

My sign shape dataset has class imbalance in the training set: 291 circular, 89 octagonal, 70 square, 44 triangular, but only 33 damaged signs. Should I use class weights in the loss function, oversample the minority class, or undersample the majority classes? What are the trade-offs for each approach?

Reflection:
The AI explained that oversampling duplicates minority samples (risks overfitting to those 33 images), while undersampling discards majority data (wastes the 291 circular samples). Class weights in CrossEntropyLoss penalize mistakes on rare classes more heavily without duplicating or discarding data, making it the best choice for my limited dataset size.

Improvements, Errors, and Contributions

Generative AI was used to obtain concise and informative overviews of different computer vision models, including their respective advantages, limitations, and suitability for specific tasks within the project. In addition, generative AI was used to generate data visualisation code, such as Matplotlib plots, which facilitated result analysis and aided in debugging and performance evaluation throughout the assignment.

Despite these benefits, several limitations were observed when using generative large language models, particularly during debugging tasks. In some cases, the models incorrectly identified specific lines of code as erroneous when those sections were in fact correct, while the true source of the issue was located elsewhere in the codebase. Such inaccuracies highlighted the risk of over-reliance on AI-generated suggestions and underscored the importance of maintaining critical oversight when integrating generative AI into the development process.

In conclusion, generative AI provided valuable assistance throughout the research and implementation phases of the project, contributing to improved efficiency and faster iteration. However, its outputs were not always reliable and required careful validation. AI-generated code and explanations must therefore be treated as supportive aids rather than definitive solutions, with manual verification remaining essential to ensure correctness and robustness.

Individual Reflection

Daniel Borg Finn

Throughout the development of the project generative AI not only saved time when dealing with bugs/errors, but also deepened my understanding on the vision techniques used. For instance, Gen AI taught me the importance of using a good balance of hyperparameters for each model, whilst also suggesting alternatives which gave greater results at the cost of factors such as training time or memory. However, despite these quick suggestions, some ideas proposed by the AI caused even more problems when it was not given enough context, which sometimes made development take longer than initially planned. In other words, these tools are highly useful at gathering resources but it is up to the user to piece everything together.

Daniela Curmi

In this assignment, AI has significantly supported my learning and implementation by providing useful definitions and giving a better understanding of the underlying concepts. For example, it helped clarify transformer-based and CNN-based model architectures, and when each model should be preferred based on the task requirements. Through interactions with Chat GPT's GPT-5.2 model, I gained deeper understanding of complex concepts in object detection and classification tasks, such as training a model, trying different parameter values, and model evaluation through basic statistics methods (i.e. plotting methods).

Nonetheless, the model still has some disadvantages where the lack of context for certain prompts generated inaccurate results. For example, I had a bug when evaluating the object detector, where my model ids and COCO ids for the class labels weren't aligned (i.e. 0-5 and 1-6). This means my detections were all being labelled wrong. When using the GPT-5.2 model it started pointing towards possible errors related to training, when in reality all I needed to do was shift my ids by 1 to align. Hence, it required selective adaptation and modification to meet the specific requirements of the task at hand. In summary, GPT-5.2 was a useful asset throughout the project, serving as a tool for brainstorming ideas and refining code.

Matthew Pirotta

Overall, generative AI improved my efficiency during this project by supporting both learning and implementation. It was particularly useful in the early stages for understanding object detection and attribute classification models and how they could be combined within a traffic sign analysis pipeline. Generative AI also enabled faster iteration when configuring training workflows, analysing results, and debugging code, allowing issues to be resolved more quickly than through trial and error alone. The ability to ask follow-up questions and receive immediate feedback was valuable when working with unfamiliar concepts or libraries. However, for deeper technical understanding and validation, I relied on official documentation and research papers, using generative AI as a supportive rather than authoritative tool.

Matthew Vella

Generative AI was most useful when I encountered unfamiliar error patterns, like the YOLO memory crash or discovering that smartphone EXIF metadata was causing my bounding box crops to extract the wrong image regions. These weren't issues I would have diagnosed quickly on my own. I found the quality of responses improved significantly when I asked targeted questions about specific design decisions rather than implementation details. Questions like whether to calculate class weights from the full dataset or only the training set, or which augmentations are suitable for traffic signs (horizontal flip acceptable, vertical flip unacceptable), produced more useful and valuable insights than asking for code. Understanding the reasoning behind approaches let me implement solutions properly rather than copy-pasting without context.

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY

Declaration

Plagiarism is defined as "the unacknowledged use, as one's own work, of work of another person, whether or not such work has been published" (Regulations Governing Conduct at Examinations, 1997, Regulation 1 (viii), University of Malta).

I / We*, the undersigned, declare that the [assignment / Assigned Practical Task report / Final Year Project report] submitted is my / our* work, except where acknowledged and referenced.

I / We* understand that the penalties for making a false declaration may include, but are not limited to, loss of marks; cancellation of examination results; enforced suspension of studies; or expulsion from the degree programme.

Work submitted without this signed declaration will not be corrected, and will be given zero marks.

* Delete as appropriate.

(N.B. If the assignment is meant to be submitted anonymously, please sign this form and submit it to the Departmental Officer separately from the assignment).

Matthew Vella
Student Name

Signature

Matthew Pirotta
Student Name

Signature

Daniela Curmi
Student Name

Signature

Daniel Borg Finn
Student Name

Signature

ARI3129
Course Code

Advanced Computer Vision for AI Course Project
Title of work submitted

24/01/2026
Date